



VPOS 2.0

Colaborando para construir tu negocio en internet

Especificaciones Técnicas
Single Buy
Versión 1.22

Control de cambios

Sección/hoja	Versión	Fecha	Descripción
Pago ocasional/Pág. 8	Versión 1.3	09/08/2018	Se agrega sección "Tarjetas procesadas"
Pago con token/Pág. 18	Versión 1.3	09/08/2018	Se agrega sección "Tarjetas procesadas"
Pagos con débito	Versión 1.3	09/08/2018	Se elimina la sección de pagos con débito.
Catastro de tarjeta/Pág. 26	Versión 1.3	09/08/2018	Se agrega la sección "Flujo de catastro"
Catastro de tarjeta/Pág. 27	Versión 1.3	09/08/2018	Se agrega "Recomendación para el comercio"
	Versión 1.4	18/09/2018	Cambio de pago anónimo por pago ocasional
Código de errores -Pag 50	Versión 1.5	15/10/2018	Anexo Código de errores
Catastro de tarjeta - Pag 22	Versión 1.6	14/01/2019	Recomendación para el comercio.
Single Buy Zimple - Pag 17	Versión 1.7	05/04/2019	Integración Zimple-vPOS
	Version 1.8	30/08/2019	Recomendación para aplicativos
Pag 51	Version 1.8	30/08/2019	Paso a producción
Pag 42	Version 1.9	11/09/2019	Reversas operativas
Pag 11	Version 1.10	25/08/2020	Se agrega nueva funcionalidad del additional_data para soportar múltiples promociones
Pag 21	Version 1.10	25/08/2020	Se agrega datos de prueba de zimple
	Versión 1.11	07/05/2021	Agregar soporte para la Ley de Servicios Digitales

Pag 10 – Flag preautorizacion en single buy Pag 35 – Flag preautorizacion en charge Pag 52 – Servicio nuevo para confirmar una preautorizacion	Version 1.12	21/05/2021	Agregar preautorizacion
	Version 1.13	11/01/2023	Se quita el flujo de tet ya no valido para vpos
Pag 33	Version 1.14	28/08/2023	Se agrega datos adicionales para el servicio de listar tarjetas
Pag 65	Versión 1.15	12/09/2023	Se agrega flujo de Preautorizacion con TC y con TD
Pag 39-40	Version 1.16	21/03/2024	Se agrega flujo para 3DS
Pág. 8	Versión 1.17	07/10/2024	Agrega soporte para TD local en Pago Ocasional
Pág. 11	Versión 1.17	07/10/2024	Agrega parámetro “extra_response_attributes”
Pág. 37	Versión 1.17	16/12/2024	Corrección del tiempo de espera de Operación de confirmación
Pág. 63	Versión 1.18	13/02/2025	Mejoras visuales sobre los formularios de catastros y pagos ocasionales.
Pag. 12 – Flag billing para single buy Pág. 13-16 – Parámetros de billing a enviar en single buy Pág. 22-24 – Parámetros de billing a enviar en single buy zimple Pág. 37-39 – Parámetros de billing a enviar en pago con token Pág. 45-47 – Parámetros a recibir en responde de pago ocasional y pago con token Pág. 66-71 – Operaciones exclusivas y consideraciones para factura electrónica	Versión 1.19	10/07/2025	Integración con Factura Electrónica
Pág. 39	Versión 1.19	10/07/2025	Se agrega aclaratoria con respecto al flujo de pago con token para comercios del rubro Casinos y juegos de azar
Pág. 72	Versión 1.19	10/07/2025	Agregadas aclaraciones con respecto a la operativa de confirmación de preautorizaciones.
Pág. 11, Pág. 21, Pág. 37	Versión 1.20	10/07/2025	Se modifica límite de campo additional_data

Pág. 13-14	Versión 1.20	10/07/2025	Se agregan opción para promociones con bins
Pág. 83-84	Versión 1.21	17/11/2025	Se actualizo la tabla "Código de errores en los pagos"
Pág 84	Versión 1.22	08/01/2026	Especificación de mecanismo de bloqueo para envíos múltiples de débito.

Contenido

Contenido	4
Introducción	5
Autenticación	7
Token.....	7
Pago ocasional	8
Operaciones pago ocasional.....	9
Single Buy (Pedido de pago)	9
Single Buy Zimple (Pedido de pago con Zimple).....	20
Catastro y Pago con token	26
Operaciones para catastro y pago con token	27
Catastro de Tarjeta (Cards_new).....	27
Recuperar Tarjetas catastradas de un usuario (users_cards).....	32
Pago con token(charge)	35
Flujo 3D SECURE Pago con token - Charge	39
Eliminar tarjeta.....	41
Operaciones comunes para pago ocasional y pago con token.....	43
Buy Single Confirm (Operación de confirmación de una transaccion)	43
Información índice de riesgos	47
Single Buy Rollback (Operación de reversa de transacción)	49
Get Buy Single Confirmation(Operación de consulta de una transacción)	54
Preauthorization Confirm(Operación de confirmación de una preautorización).....	60
Operaciones Exclusivas con Facturas Electrónicas.....	66
Get Client Info by RUC (Operación para obtener datos de cliente para Factura Electrónica)	66

Cancel Generated Invoice (Operación para cancelar Factura Electrónica)	68
Flujo de una Preautorización:	71
Flujo con Tarjeta de crédito:	71
Observación:	71
Flujo con Tarjeta de débito:	72
Observación:	72
Restricciones del comercio	73
Solicitud de pase a producción	74
Mejoras visuales sobre los formularios de catastros y pagos ocasionales	75
Transición de los formularios a los nuevos estilos	75
Formulario de catastro	76
Formulario de pagos ocasionales	76
Nuevas opciones de personalización	77
Vista previa	79
Temas predefinidos	80
Código de errores – Vpos 2.0	81

Introducción

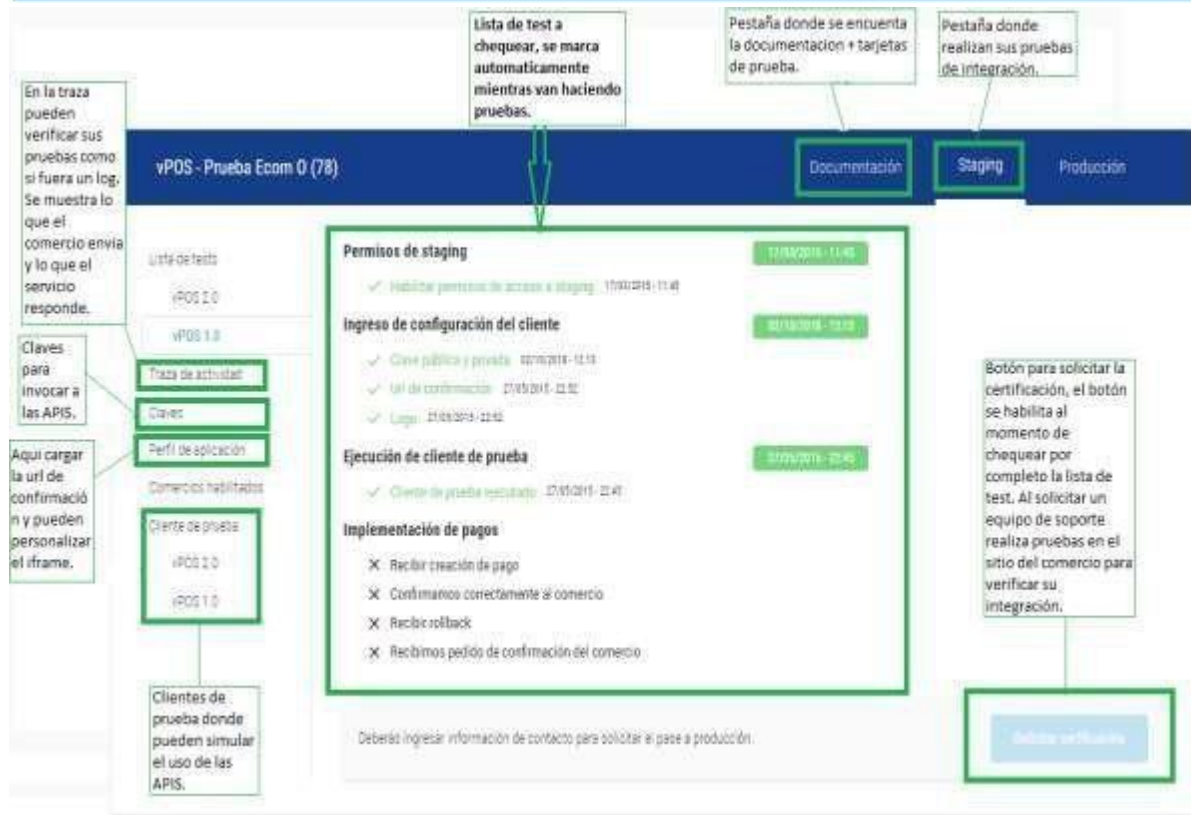
El siguiente documento presenta la información técnica necesaria para comunicarse con el servicio de pasarela de pagos de eCommerce de Bancard o VPOS.

El producto por construir por el comercio podrá ser Web o Mobile. A continuación, se detallan las distintas interacciones con servicios de la API REST, así como redirecciones necesarias a una interfaz de Bancard para solicitar los datos de la tarjeta de crédito.

Adicionalmente a este documento el comercio o desarrollador de la integración con VPOS deberá contar con un acceso al portal de comercio de Bancard: <https://comercios.bancard.com.py>

En el mismo se le brindará acceso para:

- Acceder al ambiente de staging y producción de vpos
- Acceder a la clave pública y privada. Adicionalmente podrá regenerar ambas claves.
- Modificar información del perfil: Nombre, logo y url de confirmación
- Traza de interacciones entre VPOS y el producto desarrollado por el comercio.
- Checklist con pasos para validar la integración.
- Documentación y ejemplos de códigos en distintos lenguajes.



El vPOS 2.0 cuenta con dos formas de pago, que son las siguientes:

- 1- **Pago ocasional:** El usuario carga siempre todos los datos de su tarjeta en el formulario realizando así el pago.

Servicios ofrecidos:

- **single_buy** - inicia el proceso de pago.

- 2- **Pago con token:** El usuario catastra su tarjeta y realiza el pago con un click. **Servicios ofrecidos:**

- **Cards_new** – Inicia el proceso de catastro de una tarjeta.
- **Users_cards** – operación que permite listar las tarjetas catastradas por un usuario.
- **Charge** – operación que permite el pago con un token.
- **Delete** - operación que permite eliminar una tarjeta catastrada.

Servicios que se utilizan tanto para pago ocasional como para pago con token:

- **single_buy_rollback** - operación que permite cancelar el pago (ocasional o con token).
- **get_single_buy_confirmation** - operación para consulta, si un pago (ocasional o con token) fue confirmado o no.

Servicios ofrecidos por el comercio

- **single_buy_confirm** - operación que será invocada por VPOS para confirmar un pago (ocasional o con token).

El cliente debe ofrecer en una URL pública y de común acuerdo un servicio mediante el cual se notificará la aprobación o cancelación de la transacción de un cliente final, además para funcionar como cliente Web Service de vPOS deberían soportar TLS1. 2..

Autenticación

La clave privada y pública permitirán identificar todas las interacciones con los servicios del eCommerce de Bancard. Estas claves serán enviadas por el producto desarrollado por el comercio en todas sus peticiones para identificarse (la clave privada **nunca** viaja en forma plana, sino *hasheada* con otra información en forma de *token*). Ambas pueden ser generadas nuevamente en caso de vulneración.

La clave pública **será única**, y de la forma: [a-zA-Z0-9] {32}. La clave privada no tiene porqué ser única (aunque seguramente lo sea), y de la forma: [a-zA-Z0-9T1] {40}.

Peticiones realizadas por el comercio a VPOS

Las peticiones serán realizadas por POST a una interfaz REST

public_key	Clave pública del comercio.
operation	Datos de la operación que se va a llevar a cabo.

```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    ...
  }
}
```

Token

El token será generado al momento de realizar la petición, dependiendo de la operación. Será siempre un md5 (32 caracteres). El orden debe ser exactamente como se indica.

single buy

md5(private_key + shop_process_id + amount + currency)

single_buy confirm

md5(private_key + shop_process_id + "confirm" + amount + currency)

single_buy get confirmation

md5(private_key + shop_process_id + "get_confirmation")

single_buy rollback

md5(private_key + shop_process_id + "rollback" + "0.00")

El token de confirm para una acción de rollback se genera usando "0.00" para amount.

Al momento de generar el token, los números deben ser transformados en cadenas, usar dos dígitos decimales y un punto (".") como separador de decimales. ej.:

token = md5("[private key]" + "3332134" + "130.00" + "130.00")

cards_new

md5(private_key + card_id + user_id + "request_new_card")

users_cards

md5(private_key + user_id + "request_user_cards")

charge

md5(private_key + shop_process_id + "charge" + amount + currency + alias_token)

delete

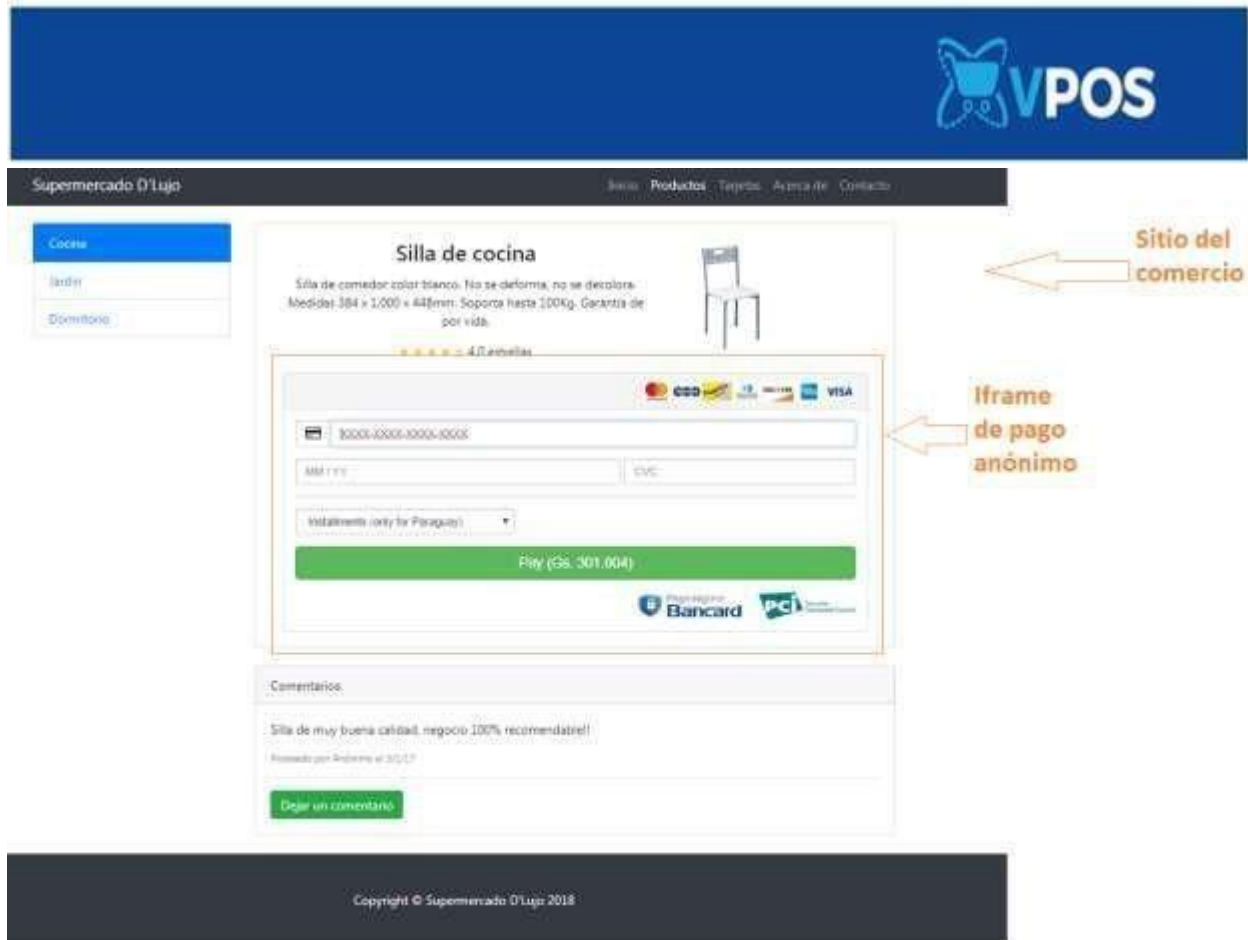
md5(private_key + "delete_card" + user_id + card_token)

Pago ocasional

Tarjetas procesadas

Esta operación acepta todas las tarjetas.

El comercio carga el iframe de pago seguro de Bancard en su sitio, donde el iframe de pago ocasional queda totalmente integrado sin necesidad de que el cliente salga del sitio del comercio.



Operaciones pago ocasional

El eCommerce de Bancard cuenta con operaciones publicadas como Web Services REST disponibles para los comercios asociados que le permitirán realizar el flujo de un carrito en su sitio.

Single Buy (Pedido de pago)

POST {environment}/vpos/api/0.3/single_buy

Environment:

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token: md5(private_key + shop_process_id + amount + currency)

Operación invocada por el comercio para iniciar el proceso de pago.

Este servicio devolverá un identificador de proceso (process id) que se utilizará para invocar el iframe de pago ocasional. Llamamos **iframe de pago ocasional** al iframe que permite cargar el formulario en el sitio del comercio.

Debe completarse con éxito un Single Buy para habilitación de la correspondiente opción en la Lista de test ->Recibir creación de pago

Obs1: **No** se marcará en la lista de test si es que en el json del pedido envían test_client.

Obs2: Si el comercio ya cuenta con el vPOS 1.0 esta operación ya lo tiene implementada, solo deben cambiar el redirect por el iframe de pago ocasional.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador de la compra.	Entero (15)
amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
iva_amount	Importe en guaraníes. Este parámetro aplica solo para aquellos comercios que deben cumplir con la “Ley de servicios digitales”	Decimal (15,2) - separador decimal ‘.’

currency	Tipo de Moneda.	String (3) - PYG (Gs)
additional_data	Campo de servicio de uso reservado para casos especiales. (ej: promociones). Opcional	String (255)
preauthorization	Campo opcional para indicar que es una preautorizacion	String (1) : S
description	Descripción del pago, para mostrar al usuario.	String (20)
return_url	URL a donde se enviará al usuario al realizar el pago. Tener en cuenta que, si la tarjeta es rechazada, también se le redirigirá a esta URL.	String (255)
cancel_url	URL a donde se enviará al usuario al cancelar el pago. Opcional, se usará return_url por defecto.	String (255)
extra_response_attributes	Parámetros para recibir datos extras en la respuesta del servicio.	Array de parámetros a enviar: payment_card_type: Ej: ["payment_card_type"] Devuelve "credit" o "debit". debit: Es una tarjeta de débito procesada por Bancard. credit: Es una tarjeta de crédito o una tarjeta no procesada por Bancard.

billing	Campo reservado para enviar información de facturación. Opcional	Billing
----------------	---	---------

Descripción de “additional_data”

Este elemento será utilizado para enviar información adicional a validar en el momento de la autorización de la compra. Se empleará para indicar promociones o convenios realizados entre el comercio, Bancard y el emisor.

Promoción Tradicional

Promoción habilitada solo para tarjetas de crédito.

La estructura de este elemento será:

Dato	Tipo de Dato	Formato	Posición	Alcance	Ejemplo
Entidad	Int (3)	Rellenado con ceros a la izquierda	1-3	TC y TD	099
Marca	String (3)	Rellenado con espacios a la derecha	4-6	TC y TD	‘VS’
Producto	String (3)	Rellenado con espacios a la derecha	7-9	TC y TD	‘ORO’
Afinidad	Int (6)	Rellenado con ceros a la izquierda	10-15	TC	000045

Ejemplo de datos a enviar si el comercio desea validar que:

. la tarjeta sea de una entidad específica: 099

- . la tarjeta sea de una entidad y afinidad específica: 099 000045
- . la tarjeta sea de una entidad y marca específica: 099VS
- . la tarjeta sea de una entidad, marca y producto específico: 099VS ORO
- . la tarjeta sea de una marca específica: 000VS
- .se puede enviar varias promociones: 099VS ORO000045,099VS,099VS ORO000045 *entre comas(,) sin espacio

Promoción con Bines

Promoción habilitada para tarjetas de crédito y débito.

En caso de que el comercio quiera indicar promociones con Bines de tarjetas, la estructura será la siguiente:

Dato	Tipo de Dato	Formato	Posición	Alcance	Ejemplo
BIN	String(13)	"BIN"<Bin de la tarjeta>	1-13	TC y TD	BIN5421002841

- . El elemento debe iniciar siempre con el String "BIN" y debe ser seguido por el bin dado
- . El elemento soporta hasta un BIN de 10 caracteres
- . Se pueden enviar varias promociones separadas entre comas sin espacios: BIN54320034,BIN544198

Observaciones:

- No habilitado para tarjetas INFONET de BIN compartido.
- No habilitado para pagos QR tarjetas de débito.
- El formato de promociones con Bines no es compatible con el formato de la promoción tradicional. El comercio deberá decidir entre uno u otro formato para indicar la promoción que desee aplicar.

Consideraciones a tener en cuenta para integrar Factura Electrónica

- El comercio debe estar habilitado para emitir Factura Electrónica.
- El campo del Timbrado (commerce_stamp) debe ser válido.

- Los campos de Establecimiento (commerce_establishment) y Punto de Expedición (commerce_expedition_point) deben corresponder al Timbrado enviado.
- Si el valor del RUC del cliente (client_ruc) es distinto a nulo o vacío, los valores del nombre (client_name) y correo electrónico (client_email) también deben ser distinto a nulo o vacío.
- Si el valor del RUC del cliente (client_ruc) es igual a nulo o vacío, se considerará como una Factura Innominada.
- Las facturas innominadas solo pueden ser generadas hasta un valor de Gs 7.000.000, según reglamentación de la DNIT.
- La lista de ítems adquiridos (details) no puede estar vacía.
- El costo total de los ítems en operation.billing.details debe coincidir con el monto enviado en operation.amount.
- La factura electrónica será emitida si y solo si los datos de facturación enviados son válidos y el pago ha sido confirmado correctamente.
- En el flujo de pre-autorización, la factura electrónica se genera luego de la confirmación de la transacción.
- En caso de que los datos de facturación enviados no sean válidos, se emitirá un mensaje de error detallando el problema, y no se procesará el pago.
- En caso de que los datos de facturación sean válidos, el pago se procese sin inconvenientes, pero la factura electrónica no se haya podido generar correctamente, se emitirá un mensaje detallando el problema con la facturación en el campo billing_response. El pago se mantendrá en estado procesado.
- Los datos de todas las facturas electrónicas emitidas estarán disponibles en el portal de facturación electrónica para ser consultadas.
- Si ha ocurrido un problema con la facturación, el comercio tendrá la opción de volver a generar la factura desde el portal de facturación electrónica.

Descripción de elemento Billing

Este elemento será utilizado para enviar la información de facturación del comercio. Los datos recibidos serán utilizados para emitir una factura electrónica, si es que el comercio está habilitado para hacerlo. Debe tener en cuenta ciertas [consideraciones](#) para la integración correcta de este elemento.

La estructura de este elemento será:

client_ruc	RUC del Cliente	String (15) Obs: Si se envía null se emitirá una factura innominada
client_name	Nombre o Razón Social del Cliente	String (100)

client_email	Correo electrónico del Cliente	String (32)
commerce_stamp	Timbrado	String (32)
commerce_expedition_point	Punto de Expedición	String (32)
commerce_establishment	Establecimiento	String (32)
details	Detalles de ítems adquiridos	Array de Details

Elemento Details:

description	Descripción del articulo adquirido	String (255)
amount	Precio unitario del articulo adquirido	Decimal (15,2) - separador decimal ‘.’
iva_rate	Tasa de IVA aplicada al producto.	Entero (15)
total_items	Cantidad del mismo artículo adquirido	Entero (15)

Ejemplo petición:

```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    "shop_process_id": 54322,
    "currency": "PYG",
    "amount": "10330.00",
    "iva_amount": "1033.00",
    "additional_data": "099VS ORO000045",
    "description": "Ejemplo de pago",
    "return_url": "http://www.example.com/finish",
    "cancel_url": "http://www.example.com/cancel",
    "billing": {
      "client_ruc": "123456-1",
      "client_name": "JUAN GONZALEZ",
      "client_email": "juangonzalez@mail.com.py",
      "commerce_stamp": "12559969",
      "commerce_expedition_point": "001",
      "commerce_establishment": "002",
      "details": [
        {
          "description": "item 1",
          "amount": "10000.00",
          "iva_rate": 10,
          "total_items": 1
        },
        {
          "description": "item 2",
          "amount": "330.00",
          "iva_rate": 10,
          "total_items": 1
        }
      ]
    }
  }
}
```

La respuesta estará compuesta por un JSON con los siguientes elementos:

status	Estado de respuesta	String (20)
process_id	Identificador de la compra	String (20)

Ejemplo respuesta:

```
{  
  "status": "success",  
  "process_id": "i5fn*Ix6niQel0QzWK1g"  
}
```

Nota:

"El mensaje de respuesta se enviará en el cuerpo (body) de la petición HTTP"

Invocar al iframe de pago ocasional

Una vez obtenido el **process_id** en la operación de **single_buy**, el usuario podrá incluir en su e-commerce un formulario de checkout embebido, de esta forma la compra se podrá finalizar en su propia aplicación. Para esto podrá utilizar la librería JavaScript como se indica en el siguiente repositorio de código.

<https://github.com/Bancard/bancard-checkout-js>

El JavaScript para iframe de pago ocasional se encuentra publicado:

`src="https://{enviroment}/checkout/javascript/dist/bancard-checkout-4.0.0.js">`

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Para levantar el iframe:

```
window.onload = function ()  
{
```

```
  Bancard.Checkout.createForm('iframe-container', process_id, styles);
```

```
};
```

Ejemplo de código html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>iFrame</title>

  <script src="https://vpos.infonet.com.py:8888/checkout/javascript/dist/bancard-checkout-1.0.0.js"></script>

</head>

<script type="application/javascript"> styles = {
  "form-background-color": "#001b60",
  "button-background-color": "#4faed1",
  "button-text-color": "#fcfcfc",
  "button-border-color": "#dddddd",
  "input-background-color": "#fcfcfc",
  "input-text-color": "#111111",
  "input-placeholder-color": "#111111"
};
window.onload = function () {
  Bancard.Checkout.createForm('iframe-container', 'WR-YY9JmxsEZV3hpVGA7', styles);
};
</script>
</html>
```

Panel de personalización

Pueden personalizar el iframe también por medio de una tabla de personalización que se encuentra en el panel de vpos del portal de comercio en el apartado de Perfil de la aplicación.

Personalización del formulario

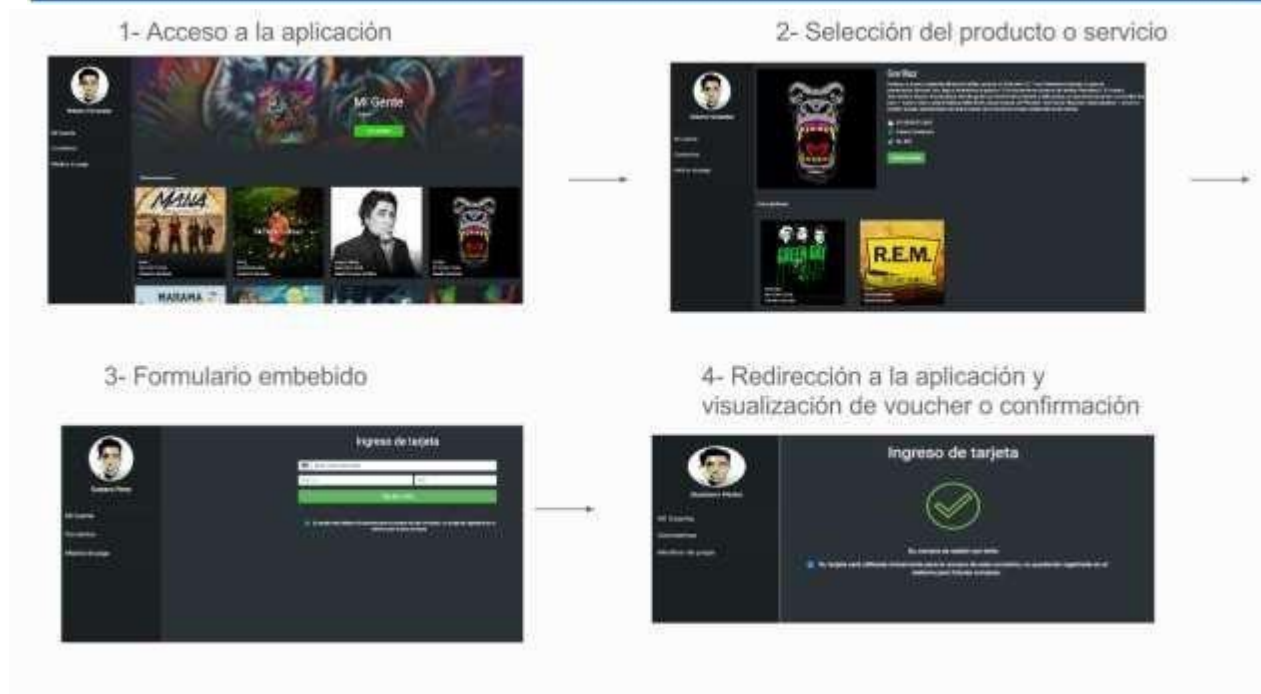
Atributo	Valor
Color fondo de campos	<input type="color" value="#FFFFFF"/>
Color texto de campos	<input type="color" value="#000000"/>
Color borde de campos	<input type="color" value="#CCCCCC"/>
Color fondo del botón	<input type="color" value="#FFA500"/>
Color texto del botón	<input type="color" value="#FFFFFF"/>
Color borde del botón	<input type="color" value="#FF4500"/>
Color fondo de formulario	<input type="color" value="#FFFFFF"/>
Color del borde del formulario	<input type="color" value="#CCCCCC"/>
Color fondo de encabezado	<input type="color" value="#FFFFFF"/>
Color texto de encabezado	<input type="color" value="#000000"/>
Color de línea separadora	<input type="color" value="#CCCCCC"/>
Color del placeholder	<input type="color" value="#000000"/>
Color texto de tu-eres-tu	<input type="color" value="#000000"/>

Guardar

Luego de que el usuario ingrese los datos de su tarjeta y le da al botón de **PAGAR**, entonces el vpos realiza un POST a la url de confirmación que el comercio proporcione en el panel de la aplicación.

Es la siguiente operación: [Operación de confirmación](#)

Experiencia de compra de un cliente en un sitio con el **iframe de pago ocasional**



Single Buy Zimple (Pedido de pago con Zimple)

POST {environment}/vpos/api/0.3/single_buy

Environment:

- Producción - <https://vpos.infonet.com.py>
- Staging - <https://vpos.infonet.com.py:8888>

Token: md5(private_key + shop_process_id + amount + currency)

Operación invocada por el comercio para iniciar el proceso de pago por zimple. Es el mismo servicio que se utiliza para el pago ocasional.

Este servicio devolverá un identificador de proceso (process id) que se utilizará para invocar el iframe de zimple. Llamamos **iframe de pago zimple** al iframe que permite cargar el formulario en el sitio del comercio.

Obs: Si tiene implementado el pago ocasional, para implementar **Zimple**, solo tiene 2 variantes, el **additional_data** y el campo **zimple**.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador de la compra.	Entero (15)
amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
currency	Tipo de Moneda.	String (3) - PYG (Gs)
additional_data	Campo donde ira el teléfono celular del usuario con Zimple.	String (255) Ej: “0981123456”
description	Descripción del pago, para mostrar al usuario.	String (20)
return_url	URL a donde se enviará al usuario al realizar el pago. Tener en cuenta que, si la tarjeta es rechazada, también se le redirigirá a esta URL.	String (255)

cancel_url	URL a donde se enviará al usuario al cancelar el pago. Opcional, se usará return_url por defecto.	String (255)
billing	Campo reservado para enviar información de facturación. Opcional	Billing
zimple	Valor que enviar cuando se quiere invocar el iframe de simple, enviar "S"	String (1) Ej: "S"

Descripción de elemento Billing

Este elemento será utilizado para enviar la información de facturación del comercio. Los datos recibidos serán utilizados para emitir una factura electrónica, si es que el comercio está habilitado para hacerlo. Debe tener en cuenta ciertas [consideraciones](#) para la integración correcta de este elemento.

La estructura de este elemento será:

client_ruc	RUC del Cliente	String (15) Obs: Si se envía null se emitirá una factura innominada
client_name	Nombre o Razón Social del Cliente	String (100)
client_email	Correo electrónico del Cliente	String (32)
commerce_stamp	Timbrado	String (32)

commerce_expedition_point	Punto de Expedición	String (32)
commerce_establishment	Establecimiento	String (32)
details	Detalles de ítems adquiridos	Array de Details

Elemento Details:

description	Descripción del articulo adquirido	String (255)
amount	Precio unitario del articulo adquirido	Decimal (15,2) - separador decimal ‘.’
iva_rate	Tasa de IVA aplicada al producto.	Entero (15)
total_items	Cantidad del mismo artículo adquirido	Entero (15)

Ejemplo petición:

```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    "shop_process_id": 54322,
    "currency": "PYG",
    "amount": "10330.00",
    "additional_data": "0981123456",
    "description": "Ejemplo de pago",
    "return_url": "http://www.example.com/finish",
    "cancel_url": "http://www.example.com/cancel",
    "zimple": "S",
    "billing": {
      "client_ruc": "123456-1",
      "client_name": "JUAN GONZALEZ",
      "client_email": "juangonzalez@mail.com.py",
      "commerce_stamp": "12559969",
      "commerce_expedition_point": "001",
      "commerce_establishment": "002",
      "details": [
        {
          "description": "item 1",
          "amount": "10000.00",
          "iva_rate": 10,
          "total_items": 1
        },
        {
          "description": "item 2",
          "amount": "330.00",
          "iva_rate": 10,
          "total_items": 1
        }
      ]
    }
  }
}
```

La respuesta estará compuesta por un JSON con los siguientes elementos:

status	Estado de respuesta	String (20)
process_id	Identificador de la compra	String (20)

Ejemplo respuesta:

```
{
  "status": "success",
  "process_id": "i5fn*Ix6niQel0QzWK1g"
}
```

Nota:

"El mensaje de respuesta se enviará en el cuerpo (body) de la petición HTTP"

Invocar al iframe de pago con zimple

Una vez obtenido el **process_id**, el usuario podrá incluir en su e-commerce un formulario de checkout embebido, de esta forma la compra se podrá finalizar en su propia aplicación.

El JavaScript para iframe de pago con zimple se encuentra publicado:

src="<https://{environment}/checkout/javascript/dist/bancard-checkout-3.0.0.js>">

Environment

- Producción - <https://vpos.infonet.com.py>
- Staging - <https://vpos.infonet.com.py:8888>

Para levantar el iframe:

```
window.onload = function () {
```

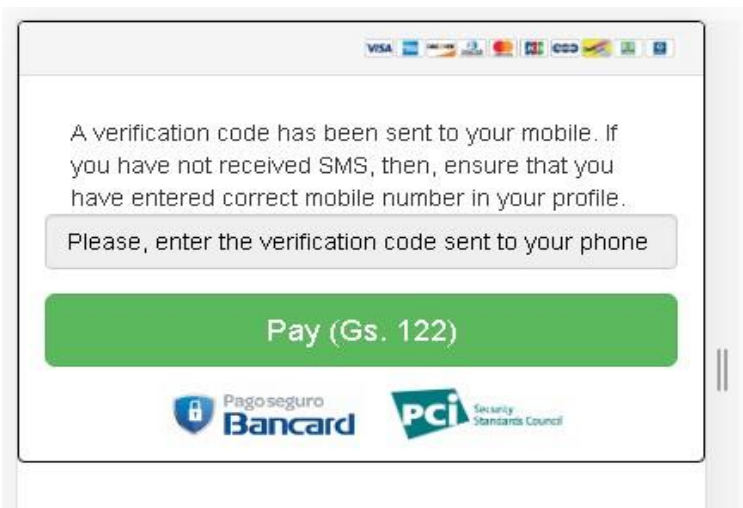
```
  Bancard.Zimple.createForm ('iframe-container', process_id ', styles);
```

```
};
```

Ejemplo de código html

Ejemplo de código html

Ejemplo de iframe Zimple



Flujo para pago con Zimple

- Enviar el pedido de single_buy con las variantes para Zimple.
- El servicio enviará un código al teléfono cargado en el campo additional_data.
- Levantar el iframe Zimple.
- El usuario debe cargar el código que llega a su teléfono en el iframe.
- Al confirmar el pago se debitará de su billetera Zimple.

Obs: El teléfono de prueba es 0981123456 y el código OTP para las pruebas es 1234 para una transacción aprobada.

Luego de que el usuario ingrese los datos de su tarjeta y le da al botón de **PAGAR**, entonces el vpos realiza un POST a la url de confirmación que el comercio proporcione en el panel de la aplicación.

Es la siguiente operación: [Buy Single Confirm \(Operación de confirmación de una transacción\)](#)

Catastro y Pago con token

Esta es una opción totalmente nueva para el comercio, donde se puede catastrar una tarjeta y realizar el pago con el token generado en el catastro.

Vpos 2.0 contará con la opción de catastro de tarjetas dentro de un **iframe de catastro** siempre en el ambiente seguro de Bancard cumpliendo con las normas PCI.

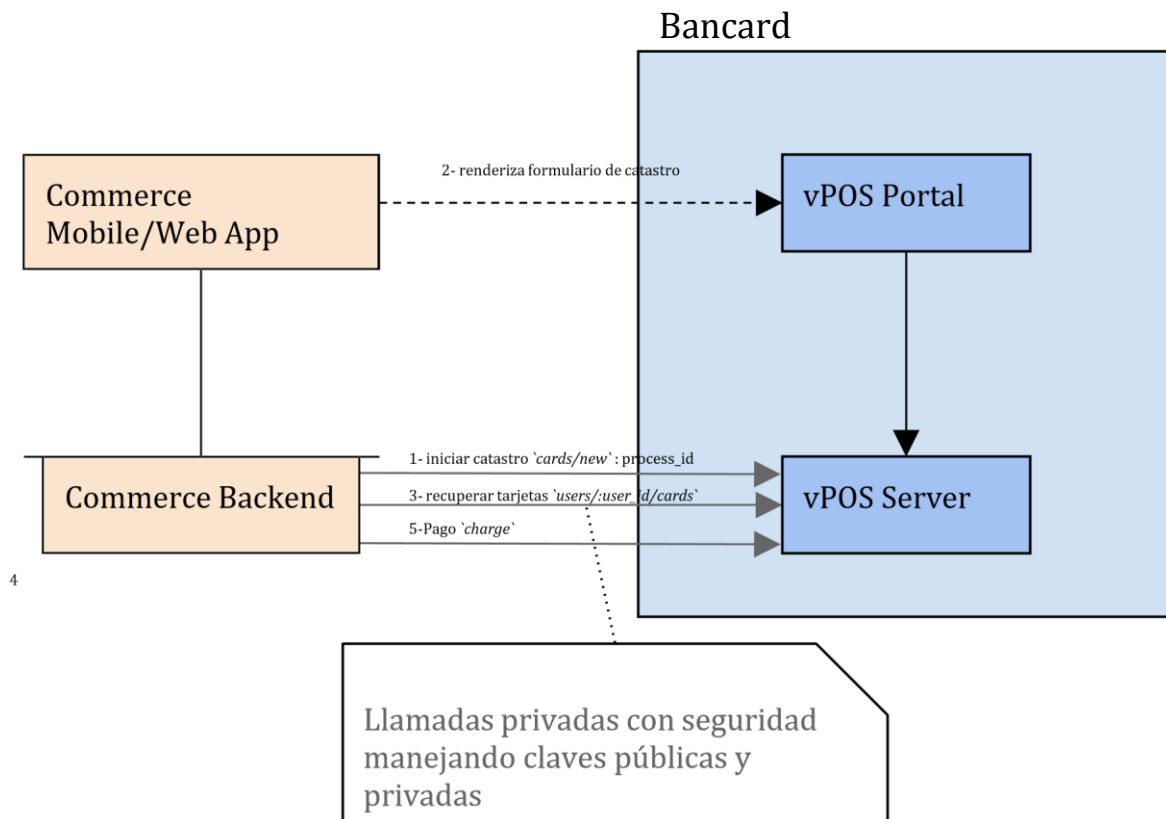
Tarjetas procesadas

Esta operación acepta:

- Tarjetas de crédito local/internacional.
- Tarjeta de débito local/internacional.

Arquitectura planteada

Se plantea un e-commerce genérico con un backend (Commerce Backend), frontend web (Commerce Web App) y mobile (Commerce Mobile App). Los comercios pueden acceder a la API Rest de vPOS (vPOS Service) y al portal de vPOS (vPOS Portal) ambos dos instalados en Bancard cumpliendo las normas PCI.



Operaciones para catastro y pago con token

Catastro de Tarjeta (Cards_new)

POST {environment}/vpos/api/0.3/cards/new

Environment:

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token:

`md5(private_key + card_id + user_id + "request_new_card")`

Operación invocada por el comercio para iniciar el proceso de catastro.

Este servicio devolverá un identificador de proceso (process id) que se utilizará para invocar el iframe de catastro. Llamamos **iframe de catastro** al iframe que permite generar un token de tarjeta para pagos con un click.

Debe completarse con éxito un Cards_new para habilitación de la correspondiente opción en la Lista de test -> Solicitud de catastro

Obs: No se marcará en la lista de test si es que en el json del pedido envían test_client.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
card_id	identificador de la tarjeta del usuario	Entero (19)
user_id	Identificador del usuario	Entero (19)
user_cell_phone	Teléfono del usuario	String (255)

user_mail	Mail del usuario	String (255)
return_url	URL a donde se enviará al usuario al realizar el pago. Tener en cuenta que, si la tarjeta es rechazada, también se le redirigirá a esta URL.	String (255)

Los atributos **card_id**, **user_id**, **user_cell_phone** y **user_mail** son obligatorios y son brindados para asociar el pedido de catastro de tarjeta a un usuario con una referencia interna del comercio.

Un usuario del comercio(**user_id**) pueden tener N tarjetas asociadas(**card_id**).

Ejemplo petición:

```
{
  "public_key": "kR6oAQoIYCqUZLAivLQgac3IO7mv5bXZ",
  "operation": {
    "token": "69bd9ef382cb47e796ebe9f6b6b850ba",
    "card_id": 1,
    "user_id": 966389,
    "user_cell_phone": 0919876543,
    "user_mail": "gustavo.rolfi@gmail.com",
    "return_url": "http://micomercio.com/resultado/catastro",
  }
}
```

La respuesta estará compuesta por un JSON con los siguientes elementos:

status	Estado de respuesta	String (20)
process_id	Identificador de la compra	String (20)

Ejemplo respuesta:

```
{
  "status": "success",
  "process_id": "i5fn*Ix6niQel0QzWK1g"
}
```

Obs: Tener en cuenta que el servicio podría devolver algún dato adicional a futuro.

Invocar al iframe de catastro de tarjeta

El usuario podrá embeber dentro de su propio sitio o app un formulario para el ingreso de información sensible de tarjetas.

Una vez que se tiene el process_id los pasos para realizar la integración son:

1. Incluir bancard-checkout.js
2. Iniciar contenedor con código JavaScript

1. Incluir bancard-checkout.js

Para utilizar la librería *bancard-checkout.js* se debe incluir la misma utilizando, por ejemplo, el siguiente código:

El JavaScript para iframe de catastro se encuentra publicado:

<https://github.com/Bancard/bancard-checkout-js>

src="**https://{enviroment}/checkout/javascript/dist/bancard-checkout-4.0.0.js**">

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

2. Iniciar contenedor con código JavaScript

Para montar el formulario de catastro en el sitio web, se debe ejecutar Bancard.Cards.createForm indicando el id del contenedor, process_id y un conjunto de opciones que incluyen los estilos asociados al elemento embebido.

Ejemplo de invocación:

```
window.onload = function () { Bancard.Cards.createForm('iframe-container','[PROCESS_ID]', styles); };
```

Ejemplo de código html

Recomendación para aplicativos que implementen catastro de tarjetas

Para comercios que implementen en su aplicativo Android tener en cuenta que para implementar el iframe se tiene que agregar las siguientes líneas de código.

```
CookieSyncManager.getInstance().startSync();
```

```
CookieManager cookieManager = CookieManager.getInstance();
```

```
cookieManager.setAcceptCookie(true);
```

```
CookieManager.getInstance().setAcceptThirdPartyCookies(registrarTarjetaWebView, true);
```

Esto es porque para el iframe necesitamos en algunas situaciones aceptar cookies y si no se tiene seteado para aceptarlas entonces el aplicativo rechaza.

Obs1: En IOS aplicativos no existe ese inconveniente.

Obs2: En Safari a partir de cierta versión se controla mediante una opción (“Prevent cross-site tracking.”) que no se pueda setear cookies desde un iFrame, en el navegador se encuentra esa opción, si desmarcan eso de su navegador ya no se da el inconveniente. Chrome también tiene esa opción, pero no viene marcado por defecto.

Flujo de catastro

Para el catastro con tarjeta de crédito:

- Se cargan los datos de la tarjeta (nro., fecha de expiración, cvv) - Se carga cedula

Para el catastro con tarjeta de débito:

- Se cargan los datos de la tarjeta (no, fecha de expiración, dato adicional) - Se carga cedula

Al momento de levantar el iframe, les aparece la opción de cargar los datos de la tarjeta y al dar siguiente les pedirá cargar un numero de cedula valido.

Tarjetas de prueba:

Nombre: MasterCard
Número: 5418630110000014
Vencimiento: 8/26
Código de seguridad: 277

Nombre: Visa
Número: 4907860500000016
Vencimiento: 8/26
Código de seguridad: 570

Nombre: Bancard
Número: 8601010000000013
Vencimiento: 8/26
Código de seguridad: N/D

La cedula válida para las tarjetas es: 9661000 (Para las pruebas)

Mensajes de respuesta del iframe de catastro

- El mensaje del iframe si se cargan correctamente los datos de la tarjeta:

```
{
  "status": " add_new_card_success ",
  "description": null
}
```

- El mensaje del iframe si no se carga correctamente los datos:

```
{
  "status": " add_new_card_fail ",
  "description": "No se ha catastrado la tarjeta. Para continuar con el catastro favor comuníquese con el CAC de Bancard. *288/4161000"
}
```

Recuperar Tarjetas catastradas de un usuario (users_cards)

POST {environment}/vpos/api/0.3/users/**user_id**/cards

Environment:

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token:

md5(private_key + user_id + "request_user_cards")

Operación invocada por el comercio para obtener las tarjetas catastradas de un usuario.

Debe completarse con éxito un `users_cards` para habilitación de la correspondiente opción en la Lista de test -> [Recibir tarjetas del usuario](#)

Obs: No se marcará en la lista de test si es que en el json del pedido envían `test_client`.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
extra_response_attributes	Parámetros para recibir datos extras en el listado de tarjetas	Array de parámetros: parámetros a enviar: <code>cards.bancard_proccesed</code> Ej: <code>["cards.bancard_proccesed"]</code> Devuelve true o false: True: Es una tarjeta procesada por Bancard False: No es una tarjeta procesada por Bancard(<code>intracountry/internacionales</code>)

El `user_id` debe ser el mismo que el comercio ingresó en la operación anterior (POST `cards/new`)

Ejemplo petición:

```
{
  "public_key": "kR6oAQoIYCqUZLAivLQgac3IO7mv5bXZ",
  "operation": {
    "token": "69bd9ef382cb47e796ebe9f6b6b850ba" ,
    "extra_response_attributes": ["cards.bancard_proccesed"]
  }
}
```

La respuesta estará compuesta por un JSON con los siguientes elementos:

status	Estado de respuesta	String (20)
cards	Elemento cards	Cards []

Array cards []

alias_token	Alias token temporal para realizar el pago	String (255)
card_masked_number	Tarjeta enmascarada	String (255)
expiration_date	Fecha espiración de la tarjeta	String (255)
card_brand	Marca de la tarjeta	String (255)
card_id	Identificador de la tarjeta	String (255)

card_type	Tipo de la tarjeta (credit o debit)	String (20)
------------------	-------------------------------------	-------------

bancard_processed	Dato que indica que la tarjeta es procesada por Bancard	Booleano: true, false Obs: este dato solo se va a recibir si el comercio envía el extra_response_attributes en el request
--------------------------	---	---

Ejemplo respuesta:

```
{
  "status": "success"
  "cards": [{
    "alias_token": "c8996fb92427ae41e4649b934ca495991b7852b855",
    "card_masked_number": "5418*****0014",
    "expiration_date": "08/21",
    "card_brand": "MasterCard",
    "card_id": 1,
    "bancard_processed": "true"
  }...]
}
```

El **alias_token** retornado permite realizar pagos con la tarjeta catastrada con la operación [Pago con token\(charge\)](#).

Es importante destacar, que el token tiene validez para una sola operación y su tiempo de vida (ttl) es del orden de los minutos.

Obs: Tener en cuenta que el servicio podría devolver algún dato adicional a futuro.

[Pago con token\(charge\)](#)

POST {environment}/vpos/api/0.3/charge

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token:

md5(private_key + shop_process_id + "charge" + amount + currency + alias_token)

El comercio podrá establecer un cargo luego de obtener las tarjetas de un usuario, para esto deberá invocar a esta operación de charge.

Debe completarse con éxito un Charge para habilitación de la correspondiente opción en la Lista de test → **Pago con alias token**

Obs: No se marcará en la lista de test si es que en el json del pedido envían test_client.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador de la compra.	Entero (15)
amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
iva_amount	Importe en guaraníes. Este parámetro aplica solo para aquellos comercios que deben cumplir con la “Ley de servicios digitales”	Decimal (15,2) - separador decimal ‘.’

currency	Tipo de Moneda.	String (3) - PYG (Gs)
number_of_payments	Cantidad de cuotas	-Débito: siempre deben enviar 1, ya que cuotas no aplica para débito. -Crédito: el comercio puede implementar un combo box donde el usuario elija la cantidad de cuotas a pagar, esto financia la entidad de la tarjeta del usuario, al comercio siempre le llega el monto total. Si envía 1 es que se realiza en un solo pago.

additional_data	Campo de servicio de uso reservado para casos especiales. (ej: promociones) Opcional	String (255)
preauthorization	Campo opcional para indicar que es una preautorizacion	String (1) : S
Alias_token	alias token obtenido de la operación de recuperar tarjetas	String (255)
extra_response_attributes	Parámetros para recibir dato para el flujo de 3DS. Siempre enviar este dato	Array de parámetros: parámetros a enviar: confirmation.process_id Ej: ["confirmation.process_id"]
Return_url	URL a donde se enviará al usuario al realizar el pago. Tener en cuenta que, si la tarjeta es rechazada, también se le redirigirá a esta URL.	String (255)
billing	Campo reservado para enviar información de facturación. Opcional	Billing

Descripción de elemento Billing

Este elemento será utilizado para enviar la información de facturación del comercio. Los datos recibidos serán utilizados para emitir una factura electrónica, si es que el comercio está habilitado para hacerlo. Debe tener en cuenta ciertas [consideraciones](#) para la integración correcta de este elemento.

La estructura de este elemento será:

client_ruc	RUC del Cliente	String (15) Obs: Si se envía null se emitirá una factura innominada
client_name	Nombre o Razón Social del Cliente	String (100)
client_email	Correo electrónico del Cliente	String (32)
commerce_stamp	Timbrado	String (32)
commerce_expedition_point	Punto de Expedición	String (32)
commerce_establishment	Establecimiento	String (32)
details	Detalles de ítems adquiridos	Array de Details

Elemento Details:

description	Descripción del articulo adquirido	String (255)
amount	Precio unitario del articulo adquirido	Decimal (15,2) - separador decimal ‘.’
iva_rate	Tasa de IVA aplicada al producto.	Entero (15)

total_items	Cantidad del mismo artículo adquirido	Entero (15)
--------------------	---------------------------------------	-------------

Flujo 3D SECURE Pago con token - Charge

Tener en cuenta que todos los comercios bajo el rubro "Casinos y juegos de azar" deberán enviar el CVV de la tarjeta para todas las transacciones. En caso de no hacerlo, se encuentra sujeto a multas por incumplimiento.

Ejemplo petición:

```
{
  "public_key": "kR6oAQoIYCqUZLAivLQgac3lO7mv5bXZ",
  "operation": {
    "token": "f9aa075da613ee2b62e6712c1ed537f2",
    "shop_process_id": 60361,
    "amount": "723215.00",
    "iva_amount": "723215.00",
    "number_of_payments": 1,
    "currency": "PYG",
    "additional_data": "",
    "description": "descripción 1",
    "return_url": "http://micomercio.com/resultado/pago3ds",
    "alias_token": "c8996fb92427ae41e4649b934ca495991b7852b855",
    "extra_response_attributes": [
      "confirmation.process_id"
    ]
  }
}
```

El alias_token es el obtenido al recuperar la lista de tarjetas de un usuario bajo el atributo con el mismo nombre.

Ejemplo respuesta: La respuesta ya viene en el response del request. El tiempo de respuesta es en segundos.

```
{
  "operation": {
    "token": "[generated token]",
    "process_id": null, // este campo vendrá null si no es un flujo 3DS
    "shop_process_id": "12313",
    "response": "S",
    "response_details": "respuesta S",
    "extended_response_description": "respuesta extendida",
    "currency": "PYG",
    "amount": 10100,
    "authorization_number": "123456",
    "ticket_number": "123456789123456",
    "iva_amount": "1100.0",
    "iva_ticket_number": "2117960079",
    "response_code": "00",
    "response_description": "Transacción aprobada."
  }
}
```

```

"security_information": {
  "customer_ip": "123.123.123.123",
  "card_source": "I",
  "card_country": "Croacia",
  "version": "0.3",
  "risk_index": "0"
}
}
}

```

Ejemplo respuesta para flujo 3DS: El flujo de 3ds devolverá todos los datos vacíos y devolverá un campo extra que es process_id para poder levantar un iframe

```

{
  "operation": {
    "token": null,
    "process_id": "i5fn*Ix6niQel0QzWK1g", //en este campo vendrá un dato si es un flujo 3DS
    "shop_process_id": null,
    "response": null,
    "response_details": null,
    "extended_response_description": null,
    "currency": null,
    "amount": null,
    "authorization_number": null,
    "ticket_number": null,
    "iva_amount": null,
    "iva_ticket_number": null,
    "response_code": null,
    "response_description": null,
    "security_information": {
      "customer_ip": null,
      "card_source": null,
      "card_country": null,
      "version": null,
      "risk_index": null
    }
  }
}
}

```

Invocar al iframe de 3D SECURE

El usuario podrá embeber dentro de su propio sitio o app.

Una vez que se tiene el process_id los pasos para realizar la integración son:

3. Incluir bancard-checkout.js
4. Iniciar contenedor con código JavaScript

3. Incluir bancard-checkout.js

Para utilizar la librería *bancard-checkout.js* se debe incluir la misma utilizando, por ejemplo, el siguiente código:

El JavaScript para iframe de catastro se encuentra publicado:

<https://github.com/Bancard/bancard-checkout-js>

`src="https://{environment}/checkout/javascript/dist/bancard-checkout-4.0.0.js">`

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

4. Iniciar contenedor con código JavaScript

Para montar el formulario de 3ds en el sitio web, se debe ejecutar `Bancard.Charge3DS.createForm` indicando el id del contenedor, `process_id` y un conjunto de opciones que incluyen los estilos asociados al elemento embebido.

Ejemplo de invocación:

```
window.onload = function () { Bancard.Charge3DS.createForm('iframe-container', '[PROCESS_ID]', styles); };
```

Ejemplo de código html

5. Luego que el iframe termine su flujo

El comercio recibirá un mensaje de success desde el iframe indicando que el flujo termino ok.

Los detalles de la transaccion el comercio recibirá en su url de confirmación, así como el día de hoy ya lo recibe cuando se procesa un pago, más información en la sección de [Buy Single Confirm \(Operación de confirmación de una transaccion\)](#)

Eliminar tarjeta

DELETE {environment}/vpos/api/0.3/users/**user_id**/cards

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token:

`md5(private_key + "delete_card" + user_id + card_token)`

Se podrá eliminar una tarjeta a un usuario, para esto se deberá invocar a la siguiente operación.

Debe completarse con éxito un delete para habilitación de la correspondiente opción en la Lista de test -> **Eliminar tarjeta del usuario**

Obs: No se marcará en la lista de test si es que en el json del pedido envían test_client.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
alias_token	alias token obtenido de la operación de recuperar tarjetas	String (255)

Ejemplo petición:

```
{
  "public_key": "kR6oAQoIYCqUZLAivLQgac3lO7mv5bXZ",
  "operation": {
    "token": "f9aa075da613ee2b62e6712c1ed537f2",
    "alias_token": "c8996fb92427ae41e4649b934ca495991b7852b855"
  }
}
```

El alias_token es el obtenido al recuperar la lista de tarjetas de un usuario bajo el atributo con el mismo nombre.

El **user_id** debe ser el mismo que el comercio ingresó en la operación de recuperar las tarjetas.

La respuesta estará compuesta por un JSON con los siguientes elementos:

status	Estado de respuesta	String (20)
---------------	---------------------	-------------

Ejemplo respuesta:

```
{  
  "status": "success"  
}
```

Obs: Tener en cuenta que el servicio podría devolver algún dato adicional a futuro.

Operaciones comunes para pago ocasional y pago con token

Buy Single Confirm (Operación de confirmación de una transacción)

POST [URL] (Definida por el comercio)

Esta acción es invocada por VPOS al finalizar una transacción. Tiene como objetivo confirmar o cancelar un pago. Este será el único medio por el cual el cliente tendrá la certeza de que el usuario completó satisfactoriamente una transacción.

Bancard realizará una petición POST a la url de confirmación que el comercio cargo en su panel de aplicación de vpos en el portal de comercios, enviando el JSON en el cuerpo del pedido o body.

El comercio deberá responder con status 200 a la operación, como se muestra más abajo en el ejemplo de respuesta. Si el comercio no responde con status 200 dentro de los siguientes 30 segundos, vPOS cerrará la conexión y se marcará como inválida la confirmación en la traza y con una indicación del timeout en reemplazo de lo que debió ser la respuesta del comercio. Si el comercio no responde con 200 eso no significa que la transacción haya quedado denegada, siempre deben realizar la consulta para verificar el estado en que quedó la transacción.

Esta operación realiza el vpos para pagos ocasionales y para pagos con token.

Nota:

Si el producto Web o Mobile desarrollado por el Comercio inicia la operación de compra (single buy) y no recibe la confirmación (single buy confirm) por parte del VPOS, puede invocar a la operación de consulta (single buy get confirmation) para saber en qué estado quedo la transacción y actualizar en su sistema o también puede invocar a la operación de reversa (single buy rollback) para evitar inconsistencias en su sistema. El tiempo de espera recomendado es de 10 minutos.

Debe completarse con éxito un Buy Single Confirm para habilitación de la correspondiente opción en la

Lista de test -> Confirmamos correctamente al comercio.

Obs1: No se marcará en la lista de test si es que en el json del pedido envían test_client.

Obs2: Si el comercio ya cuenta con el vPOS 1.0 ya está preparado para recibir la respuesta de los pagos por la url de confirmación cargada en el perfil de la aplicación del comercio.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

operation	elemento Operation	Elemento
------------------	--------------------	----------

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador interno del comercio	Entero (15)
response	Indicador de detalle procesado	String (1) - S o N
response_details	Descripción del proceso	String (60)

amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
iva_amount	Importe en guaraníes. Este parámetro aplica solo para aquellos comercios que deben cumplir con la “Ley de servicios digitales”	Decimal (15,2) - separador decimal ‘.’
currency	Tipo de Moneda.	String (3) - PYG (Gs)
authorization_number	Código de autorización.	String (6) - Solo si la transacción es aprobada.
ticket_number	Identificador de autorización.	Int (15)
response_code	Código de respuesta de la transacción.	String (2) - 00 (transacción aprobada) - 05 (Tarjeta inhabilitada) - 12 (Transacción inválida) - 15 (Tarjeta inválida) - 51 (Fondos insuficientes)
response_description	Descripción de la respuesta de transacción.	String (40)
extended_response_description	Descripción extendida de la respuesta de transacción.	String (100)
security_information	Elemento SecurityInformation	Elemento
billing_response	Elemento BillingResponse	BillingResponse

Elementos SecurityInformation

card_source	Local o Internacional	String (1) - L (Local) - I (Internacional)
customer_ip	Ip del cliente que ingresa los datos de pago	String (15)
card_country	País de origen de la tarjeta	String (30)
version	Version de la API	String (5)
risk_index	Indicador de riesgo.	Int (1)

Elementos BillingResponse

status	Resultado de la operación.	String (20): . success . error
description	Descripción del resultado de la operación	String (255)
data	Datos adicionales de la factura electrónica generada	DataBilling

Elementos DataBilling

invoice_number	Numero de la factura generada	String(32)
-----------------------	-------------------------------	------------

Ejemplo petición:

```
{
  "operation": {
    "token": "[generated token]",
    "shop_process_id": "12313",
    "response": "S",
    "response_details": "respuesta S",
    "extended_response_description": "respuesta extendida",
    "currency": "PYG",
    "amount": "10100.00",
    "authorization_number": "123456",
    "ticket_number": "123456789123456",
    "iva_amount": "1100.0",
    "iva_ticket_number": "2117960079",
    "response_code": "00",
    "response_description": "Transacción aprobada.",
    "security_information": {
      "customer_ip": "123.123.123.123",
      "card_source": "I",
      "card_country": "Croacia",
      "version": "0.3",
      "risk_index": "0"
    },
    "billing_response": {
      "status": "success",
      "description": "Factura generada correctamente",
      "data": {
        "invoice_number": "001-001-0002563"
      }
    }
  }
}
```

Notas:

Información índice de riesgos

- El atributo de “risk_index”

Consiste en un índice de riesgo de la transacción en tiempo real, este campo devolverá un número que indicará al comercio el riesgo de la transacción en tiempo real de acuerdo con la siguiente tabla:

Escala	Riesgo
0	No se puede generar el riesgo en tiempo real
1	Bajo
2	Bajo
3	Bajo
4	Medio
5	Medio
6	Medio
7	Alto
8	Alto
9	Alto

El **índice de riesgo** será generado para las transacciones que se realicen con **tarjeta de crédito local**.

Para las transacciones con tarjetas internacionales el campo risk_index mostrará 0.

Para las transacciones con tarjetas de débito el campo risk_index mostrará 0.

Para las transacciones con tarjetas de crédito de otra procesadora (cabal, panal) mostrará 0.

El campo risk_index mostrará 0 cuando no se puede generar el índice de riesgo en tiempo real.

Acciones del comercio:

- Para una transacción con Riesgo Bajo, el comercio puede estar tranquilo con la transacción.
- Para una transacción con Riesgo Medio, el comercio puede pedir datos de seguridad al cliente para verificar si la transacción le corresponde.
- Para una transacción con Riesgo Alto, el comercio debe verificar la transacción con el cliente y llamar a Bancard en caso de no tener respuesta del cliente, puede escribir un correo a riesgos@bancard.com.py y asegurar la transacción. En caso de que el comercio tenga que entregar una mercadería, favor primero verificar con Bancard si es una transacción legítima.

Ejemplo respuesta esperada por el comercio:

```
{  
  "status": "success"  
}
```

Single Buy Rollback (Operación de reversa de transacción)

POST {environment}/vpos/api/0.3/single_buy/rollback

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888>

Token: md5(private_key + shop_process_id + "rollback" + "0.00")

Esta operación se puede utilizar para pago ocasional, para pagos con token y preautorizaciones confirmadas y también para cancelar una preautorización que todavía no se confirmó.

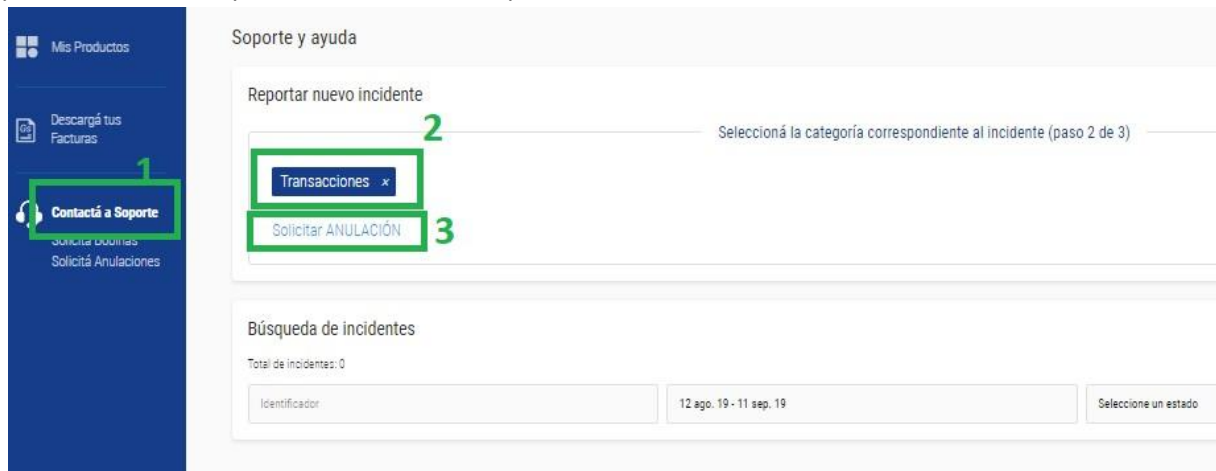
Si una preautorización no se reversa, la misma vence en 30 días y el monto se le devuelve al usuario.

La operación de Rollback deberá ser enviada en los siguientes casos:

- Para realizar un reverso de pago.
- Para transacciones canceladas por el usuario en la página de vPos.
- Para transacciones abandonadas o no culminadas por el usuario.
- Para cancelar una preautorización. Si la preautorización no se cancela la misma expira en 30 días

La operación de rollback solo puede enviar en el día en el que se realizó la operación, a esto lo denominamos reversas automáticas, las que se aplican antes que impacte en el extracto del cliente final.

Si quieren reversar una operación que ya impacto en el extracto, deben ingresar su pedido de anulación por el canal oficial, portal de comercios/soporte/anulaciones:



La operación del Rollback será satisfactoria mientras la transacción no haya sido CUPONADA (confirmada en el extracto del cliente). Si el JSON devuelve status: error y key: "TransactionAlreadyConfirmed", el comercio deberá realizar el proceso manual de pedido de reversión de una transacción cuponada a tramitar en el Área Comercial de Bancard.

El rollback devolverá un estado general "status". "success" indica que el pedido será notificado para cancelar. "error" indica que por alguna razón el pedido no puede continuar. Las posibles causas de error son:

- **InvalidJsonError** - Error en el JSON enviado
- **UnauthorizedOperationError** - Las credenciales enviadas no tienen permiso para la operación rollback.
- **ApplicationNotFoundError** - No existen permisos para las credenciales enviadas.
- **InvalidPublicKeyError** - Existe un error sobre la clave pública enviada.
- **InvalidTokenError** - El token se generó en forma incorrecta.
- **InvalidOperationError** - El JSON enviado no es válido. No cumple con los tipos o limites definidos.
- **BuyNotFoundError** - No existe el proceso de compra seleccionado
- **PaymentNotFoundError** - No existe un pedido de pago para el proceso seleccionado. Esto quiere decir que el cliente no pagó este pedido y deberá tomarse como una respuesta correcta para dichas situaciones.
- **AlreadyRollbackedError** - Ya existe un pedido de rollback previo.

- **PosCommunicationError** - Existen problemas de comunicación con el componente de petición de rollback. ●
- **TransactionAlreadyConfirmed** - Transacción Cuponada (Confirmada en el extracto del cliente)

En el caso de que una compra sea iniciada por el producto desarrollado por el comercio, pero no se finalice por el usuario o no se obtenga respuesta de parte de vpos luego de 10 minutos, se debería invocar un **Get Buy Single Confirmation** para conocer el estado del pedido. Si el pago todavía no ha sido realizado, el comercio puede optar por realizar un rollback del pedido invocando a la operación **Single Buy Rollback**.

Nota:

Debe completarse con éxito un **Single Buy Rollback** manual en un caso de transacción aprobada para habilitación de la correspondiente opción en la Lista de test ->**Recibir rollback**

Obs1: No se marcará en la lista de test si es que en el json del pedido envían test_client.

Obs2: Si el comercio ya cuenta con el vPOS 1.0 esta operación ya lo tiene implementada.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública	String
operation	Elemento Operation	Elemento

Elemento Operation

token	md5 de la petición	String
--------------	--------------------	--------

shop_process_id	Identificador interno del comercio	String
------------------------	------------------------------------	--------

Ejemplo petición:

<pre>{ "public_key": "[public key]", "elements": {}, "operation": { "token": "[generated token]", "shop_process_id": "12313" } }</pre>
--

La respuesta estará compuesta por un JSON con los siguientes elementos:

Elementos de la respuesta

status	Estado de respuesta	String <u>Valores posibles:</u> - success - error
messages	Array de elemento Message	Array

Elemento Message

key	Clave de respuesta	String <u>Valores posibles:</u> - InvalidJsonError - UnauthorizedOperationError - ApplicationNotFoundError - InvalidPublicKeyError - InvalidTokenError - InvalidOperationError - BuyNotFoundError - PaymentNotFoundError
		- AlreadyRollbackedError - PosCommunicationError - RollbackSuccessful - TransactionAlreadyConfirmed
level	Nivel de despliegue del mensaje	String <u>Valores posibles:</u> - info - error
dsc	Descripción de respuesta	String

Ejemplo respuesta:

```
{
  "status": "success",
  "messages": [
    {
      "key": "RollbackSuccessful",
      "level": "info",
      "dsc": "Rollback correcto."
    }
  ]
}
```

Get Buy Single Confirmation(Operación de consulta de una transacción)

POST {environment}/vpos/api/0.3/single_buy/confirmations

Environment

- **Producción** - https://vpos.infonet.com.py
- **Staging** - https://vpos.infonet.com.py:8888 Token: md5(private_key + shop_process_id + "get_confirmation")

Esta acción es invocada por el comercio para consultar si existió o no una confirmación.

Debe completarse con éxito un Get Buy Single Confirm para habilitación de la correspondiente opción en la Lista de test -> Recibimos pedido de confirmación del comercio

Obs1: No se marcará en la lista de test si es que en el json del pedido envían test_client.

Obs2: Si el comercio ya cuenta con el vPOS 1.0 esta operación ya lo tiene implementada.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
------------	----------------	-------------

operation	Elemento Operation	Operation
------------------	--------------------	-----------

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador de la compra.	Entero (15)

La respuesta estará compuesta por un JSON con los siguientes elementos:

Elementos de la respuesta

status	Estado de respuesta	String <u>Valores posibles:</u> - success - error
confirmation	Información de confirmación	Elemento SingleBuyConfirmation
messages	Array de elemento Message	Array

Elemento Message

key	Clave de respuesta	String <u>Valores posibles:</u> - InvalidJsonError - UnauthorizedOperationError - ApplicationNotFoundError - BuyNotFoundError - InvalidPublicKeyError - InvalidTokenError - InvalidOperationError - PaymentNotFoundError - AlreadyRollbackedError
level	Nivel de despliegue del mensaje	String <u>Valores posibles:</u> - info - error
dsc	Descripción de respuesta	String

Elemento SingleBuyConfirmation

token	MD5 de la petición	String (32)
shop_process_id	Identificador interno del comercio	Entero (15)

response	Indicador de detalle procesado	String (1) - S o N
response_details	Descripción del proceso	String (60)
amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
currency	Tipo de Moneda.	String (3) - PYG (Gs)
authorization_number	Código de autorización.	String (6) - Solo si la transacción es aprobada.
ticket_number	Identificador de autorización.	Int (15)
response_code	Código de respuesta de la transacción.	String (2) - 00 (transacción aprobada) - 05 (Tarjeta inhabilitada) - 12 (Transacción inválida) - 15 (Tarjeta inválida) - 51 (Fondos insuficientes)
response_description	Descripción de la respuesta de transacción.	String (40)
extended_response_description	Descripción extendida de la respuesta de transacción.	String (100)
security_information	Elemento SecurityInformation	Elemento

Elementos SecurityInformation

card_source	Local o Internacional	String (1) - L (Local) - I (Internacional)
customer_ip	Ip del cliente que ingresa los datos de pago	String (15)
card_country	País de origen de la tarjeta	String (30)
version	Version de la API	String (5)
risk_index	Indicador de riesgo.	Int (1)

Ejemplo petición:

```
{  
  "public_key": "[public key]",  
  "operation": {  
    "token": "[generated token]",  
    "shop_process_id": "12313"  
  }  
}
```

Ejemplo respuesta:

```
{  
  
  "status": "success"  
  
  "confirmation": {  
  
    "token": "[generated token]",  
  
    "shop_process_id": "12313",  
    "response": "S",  
  
    "response_details": "respuesta S",  
  
    "extended_response_description": "respuesta extendida",  
  
    "currency": "PYG",  
  
    "amount": "10100.00",  
  
    "authorization_number": "123456",  
  
    "ticket_number": "123456789123456",  
  
    "response_code": "00",  
  
    "response_description": "Transacción aprobada.",  
  
    "security_information": {  
  
      "customer_ip": "123.123.123.123",  
  
      "card_source": "I",  
  
      "card_country": "Croacia",  
  
      "version": "0.3",  
  
      "risk_index": "0"  
    }  
  }  
}  
  
}
```

Preauthorization Confirm(Operación de confirmación de una preautorización)

POST {environment}/vpos/api/0.3/preauthorizations/confirm

Environment

- **Producción** - https://vpos.infonet.com.py
- **Staging** - https://vpos.infonet.com.py:8888 Token:
md5(private_key + shop_process_id + "pre-authorization-confirm")

Operación invocada por el comercio para realizar la confirmación de una preautorización.

Debe ser invocada cuando el comercio confirme la aprobación de una preautorización.

En caso de que ocurra una falla en la comunicación, y el portal no esté seguro de si la confirmación se realizó o no, deberá reintentar enviando una nueva petición de confirmación.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
shop_process_id	identificador de la compra.	Entero (15)

amount	Importe en guaraníes. Dato opcional si es que desea confirmar por otro monto que no sea el preautorizado	Decimal (15,2) - separador decimal ‘.’
billing	Campo reservado para enviar información de facturación. Dato opcional si es que desea confirmar por otro monto que no sea el preautorizado	Billing

Elemento Billing

details	Detalles de ítems adquiridos	Array de Details
----------------	------------------------------	------------------

Elemento Details:

description	Descripción del articulo adquirido	String (255)
amount	Precio unitario del articulo adquirido	Decimal (15,2) - separador decimal ‘.’
iva_rate	Tasa de IVA aplicada al producto.	Entero (15)
total_items	Cantidad del mismo artículo adquirido	Entero (15)

La respuesta estará compuesta por un JSON con los siguientes elementos:

Elementos de la respuesta

status	Estado de respuesta	String <u>Valores posibles:</u> - success - error
confirmation	Información de confirmación	
messages	Array de elemento Message	Array

Elemento Message

key	Clave de respuesta	String <u>Valores posibles:</u> - InvalidJsonError - UnauthorizedOperationError - ApplicationNotFoundError - BuyNotFoundError - InvalidPublicKeyError - InvalidTokenError - InvalidOperationError - PaymentNotFoundError - AlreadyRollbackedError
level	Nivel de despliegue del mensaje	String <u>Valores posibles:</u> - info - error

dsc	Descripción de respuesta	String
------------	--------------------------	--------

Elemento SingleBuyConfirmation

token	MD5 de la petición	String (32)
shop_process_id	Identificador interno del comercio	Entero (15)
response	Indicador de detalle procesado	String (1) - S o N
response_details	Descripción del proceso	String (60)
amount	Importe en guaraníes.	Decimal (15,2) - separador decimal ‘.’
currency	Tipo de Moneda.	String (3) - PYG (Gs)
authorization_number	Código de autorización.	String (6) - Solo si la transacción es aprobada.
ticket_number	Identificador de autorización.	Int (15)
response_code	Código de respuesta de la transacción.	String (2) - 00 (transacción aprobada) - 05 (Tarjeta inhabilitada) - 12 (Transacción inválida) - 15 (Tarjeta inválida) - 51 (Fondos insuficientes)

response_description	Descripción de la respuesta de transacción.	String (40)
extended_response_description	Descripción extendida de la respuesta de transacción.	String (100)
security_information	Elemento SecurityInformation	Elemento

Elementos SecurityInformation

card_source	Local o Internacional	String (1) - L (Local) - I (Internacional)
customer_ip	Ip del cliente que ingresa los datos de pago	String (15)
card_country	País de origen de la tarjeta	String (30)
version	Version de la API	String (5)
risk_index	Indicador de riesgo.	Int (1)

Ejemplo petición:


```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    "shop_process_id": "12313",
    "amount": 20000,
    "billing": {
      "details": [
        {
          "description": "item 1",
          "amount": "10000.00",
          "iva_rate": 10,
          "total_items": 1
        },
        {
          "description": "item 2",
          "amount": "5000.00",
          "iva_rate": 5,
          "total_items": 2
        }
      ]
    }
  }
}
```

Ejemplo respuesta:

```
{
  "status": "success",
  "confirmation": {
    "token": "[generated token]",
    "shop_process_id": "12313",
    "response": "S",
    "response_details": "respuesta S",
    "extended_response_description": "respuesta extendida",
    "currency": "PYG",
    "amount": "10100.00",
    "authorization_number": "123456",
```

```
{
  "ticket_number": "123456789123456",
  "response_code": "00",
  "response_description": "Transacción aprobada.",
  "security_information": {
    "customer_ip": "123.123.123.123",
    "card_source": "I",
    "card_country": "Croacia",
    "version": "0.3",
    "risk_index": "0"
  }
}
```

Operaciones Exclusivas con Facturas Electrónicas

Get Client Info by RUC (Operación para obtener datos de cliente para Factura Electrónica)

POST {environment}/vpos/api/0.3/billing/client_info

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888> Token:

md5(private_key + "billing_client_info")

Esta acción es invocada por el comercio para consultar la información del cliente dado su RUC. Con este servicio el comercio tendrá la posibilidad de obtener los datos de nombre o razón social y correo electrónico (campos obligatorios para la generación de la factura electrónica) de sus clientes, dado el RUC de los mismos.

Si dado el RUC de un cliente determinado, la consulta al servicio no responde con datos, o dichos datos son incorrectos/desactualizados, los valores del cliente enviados para la emisión de la factura serán utilizados para actualizar los datos del cliente, de manera que en la próxima consulta, el servicio responda con los datos actualizados.

La implementación de esta operación es opcional.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elementos Operation

token	md5 de la petición	String (32)
client_ruc	RUC del cliente	String (32)

La respuesta estará compuesta por un JSON con los siguientes elementos:

Elementos de la respuesta

status	Estado de respuesta	String <u>Valores posibles:</u> - success - error
client	Elemento Client	Client

Elemento Client

name	Nombre o razón social del cliente	String (100)
email	Correo electrónico del cliente	String (32)

Ejemplo petición:

```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    "client_ruc": " 123456-7"
  }
}
```

Ejemplo respuesta:

```
{
  "status": "success",
  "client": {
    "name": "JUAN GONZALEZ",
    "email": "juangonzalez@gmail.com"
  }
}
```

Cancel Generated Invoice (Operación para cancelar Factura Electrónica)

POST {environment}/vpos/api/0.3/billing/cancel

Environment

- **Producción** - <https://vpos.infonet.com.py>
- **Staging** - <https://vpos.infonet.com.py:8888> Token:

md5(private_key + shop_process_id + "billing_cancel")

Esta acción es invocada por el comercio para cancelar una factura electrónica generada.

La cancelación devolverá un estado general "status". "success" indica que el pedido será notificado para cancelar. "error" indica que por alguna razón el pedido no puede continuar. Las posibles causas de error son:

- **CancelInvoiceError** – Error al procesar la cancelación

El detalle del error, si hubiere, estará detallado en el campo "dsc".

Consideraciones a tener en cuenta:

- Para la cancelación, es necesario que el comprobante haya sido previamente aprobado.
- Las cancelaciones estarán permitidas hasta 24 hs después de haber sido emitidas.

La implementación de esta operación es opcional.

El pedido estará compuesto por un JSON con los siguientes elementos:

Elementos de la petición

public_key	Clave pública.	String (50)
operation	Elemento Operation	Operation

Elemento Operation

token	md5 de la petición	String
shop_process_id	Identificador interno del comercio	String

Ejemplo petición:

```
{
  "public_key": "[public key]",
  "operation": {
    "token": "[generated token]",
    "shop_process_id": "12313"
  }
}
```

La respuesta estará compuesta por un JSON con los siguientes elementos:

Elementos de la respuesta

status	Estado de respuesta	String <u>Valores posibles:</u> - success - error
messages	Array de elemento Message	Array

Elemento Message

key	Clave de respuesta	String - CancelInvoiceError - CancelInvoiceSuccessful
level	Nivel de despliegue del mensaje	String <u>Valores posibles:</u> - info - error
dsc	Descripción de respuesta	String

Ejemplo respuesta:

```
{
  "status": "success",
  "elements": {},
  "messages": [
    {
      "key": " CancelInvoiceSuccessful ",
      "level": "info",
      "dsc": "Cancelación correcta de factura electrónica."
    }
  ]
}
```

Flujo de una Preautorización:

Preautorización cuenta con dos flujos distintos, uno para Tarjeta de Crédito (TC) y otro para tarjeta de Débito (TD)

Flujo con Tarjeta de crédito:

1. El comercio envía la Preautorización por un monto X (ejemplo: 100.000 Gs).
2. Dicho Monto se congela en la cuenta del cliente, es decir, el comercio aun no recibe el dinero, el usuario ve el débito, pero este no impacta aun en la cuenta del comercio.
3. El comercio envía la confirmación de la Preautorización.

Observación:

- Si el monto es menor al Preautorizado (ejemplo: se confirma por 90.000 Gs), se realiza un cálculo de la diferencia ($100.000 \text{ Gs} - 90.000 \text{ Gs} = 10.000 \text{ Gs}$), el resultado de dicha diferencia se acredita nuevamente al usuario y el monto confirmado es el que pasa a la cuenta del comercio.
- Si el monto es mayor al Preautorizado, solo se acepta hasta un 20% mayor (ejemplo: se confirma por 110.00 Gs), en estos casos se realiza un debito adicional al usuario por la diferencia entre el monto confirmado y el monto preautorizado ($110.000 \text{ Gs} - 100.000 \text{ Gs} = 10.000 \text{ Gs}$).
- En el caso de que el comercio envíe la cancelación en vez de la confirmación, el monto inicial se descongela de la cuenta del usuario, es decir, el monto que inicialmente se preautorizó se le vuelve a habilitar al usuario para su uso normal.
- Si no se realiza la confirmación a los 30 días de la preautorización, la misma se cancela de manera automática y se devuelve la plata al usuario.
- El comercio puede cancelar la Preautorización invocando la Api mencionada más arriba.
- En caso de que falle la confirmación, el comercio puede cancelar la preautorización sin la necesidad de esperar a que esta se cancele a los 30 días.
- Todo el Flujo de la Preautorización con TC se hace bajo la misma boleta Bancard.

- Una preautorización solo puede ser confirmada una vez y no es posible hacer el reintento de la confirmación. Es decir, si una confirmación queda rechazada entonces no puede volver a ser reintentada.
 - La confirmación es una operación fundamental para la preautorización y solo al obtener la confirmación aprobada se considera la transacción como finalizada correctamente. Entonces el comercio debe de esperar a tener la confirmación aprobada para la entrega de cualquier mercadería o servicio.
4. Se realiza el flujo transaccional y se acredita en la cuenta del comercio, esto no es en línea debido a que sigue el flujo de una transacción con Tarjeta de crédito.

Flujo con Tarjeta de débito:

1. El comercio envía la Preautorización por un monto X (ejemplo: 100.000 Gs).
2. A diferencia de la preautorización con TC, en este caso al enviar la misma ya se realiza el movimiento de dinero, es decir, se acredita el monto preautorizado en la cuenta del comercio.
3. El comercio envía la confirmación de la Preautorización.

Observación:

- Si el monto es menor al Preautorizado (ejemplo: se confirma por 90.000 Gs), se realiza un cálculo de la diferencia ($100.000 \text{ Gs} - 90.000 \text{ Gs} = 10.000 \text{ Gs}$), el resultado de dicha diferencia se acredita nuevamente al usuario y el monto confirmado es el que queda en la cuenta del comercio.
- En preautorización con TD no es posible enviar montos mayores.
- En el caso de que el comercio envíe la cancelación en vez de la confirmación, se realiza una transacción a la inversa para devolver la plata al usuario, es decir, se hace una transferencia de la cuenta del comercio a la cuenta del usuario.
- Si no se realiza la confirmación a los 30 días de la preautorización, la misma se cancela de manera automática y se devuelve la plata al usuario.
- El comercio puede cancelar la Preautorización invocando la Api mencionada más arriba.
- En caso de que falle la confirmación, el comercio puede cancelar la preautorización sin la necesidad de esperar a que esta se cancele a los 30 días.
- El flujo de la preautorización con TD se realizan en boletas bancard distintas, es decir, se genera una boleta para la preautorización y confirmación.
- Una preautorización solo puede ser confirmada una vez y no es posible hacer el reintento de la confirmación. Es decir, si una confirmación queda rechazada entonces no puede volver a ser reintentada.
- La confirmación es una operación fundamental para la preautorización y solo al obtener la confirmación aprobada se considera la transacción como finalizada correctamente. Entonces el comercio debe de esperar a tener la confirmación aprobada para la entrega de cualquier mercadería o servicio.

4. Luego de enviar la confirmación y que la misma quede aprobada, finaliza el flujo, la acreditación es en línea, debido a que se trata de una transacción con TD.

Restricciones del comercio

A continuación, se presentan restricciones que debe contemplar el comercio que desarrolle la integración con el eCommerce de Bancard.

Interfaz de respuesta

Luego de que el usuario ingresa sus datos de tarjeta y se confirma al comercio por medio de la operación “Buy Single Confirm” el comercio debe desplegar una interfaz de respuesta con la aprobación de la transacción.

En esta interfaz se deben respetar las siguientes restricciones:

- Se deben indicar los datos de la transacción:

Fecha y Hora

Número de pedido (shop_process_id)

Importe (amount)

Descripción de la Respuesta (response_description)

- **No** debe mostrarse al usuario:

Código de autorización (authorization_number)

Código de respuesta (response_code)

Respuesta extendida (extended_response_description)

Información de seguridad (security_information)

Notas generales

- El Comercio debe incluir en su aplicación la sección de **Contacto**, de manera que el cliente pueda evacuar consultas referentes a las compras del ecommerce.
- La aplicación podrá registrar todos datos del cliente que requiera el comercio, salvo todos aquellos que se relacionen a sus tarjetas de crédito (Número de tarjeta, código de seguridad, vencimiento, etc.)

- El logo para utilizar por el comercio debe idealmente tener un ancho de 173 píxeles y un alto 55 píxeles. El ancho mínimo es de 85 píxeles.
- Para la comunicación con nuestro web en ambiente de desarrollo deben tener habilitado el puerto 8888.

Formato de mensajería – JSON

Para enviar y recibir información se empleará el formato JSON (JavaScript Object Notation).

Información sobre JSON - <http://json.org/>

Validación de JSON - <http://jsonlint.com/>

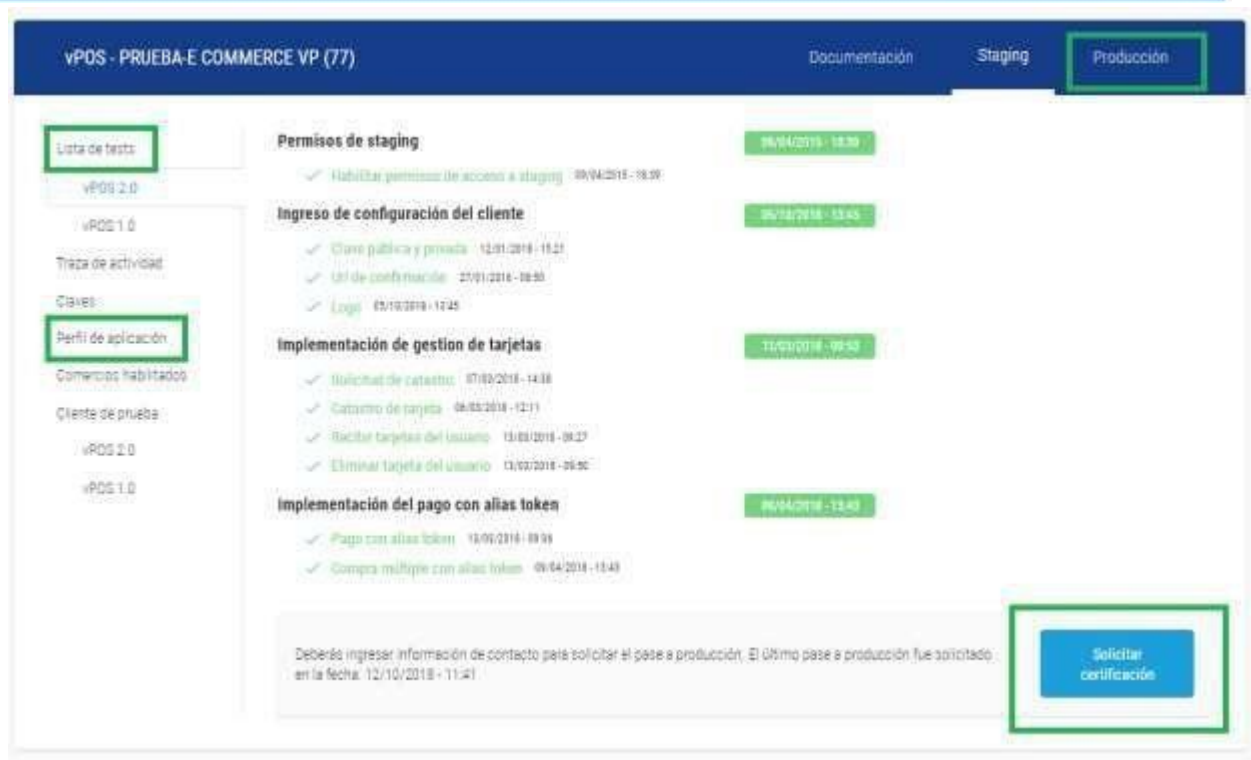
Al consumir un JSON enviado por el VPOS debe prestarse especial atención a los caracteres especiales (ej. tildes). El mismo será enviado utilizando el standard “\uXXXX” (Donde X es un dígito hexadecimal)

Los JSON enviados y recibidos por Bancard y el comercio deberán realizarse mediante una petición POST enviando el JSON en el cuerpo del pedido o body.

Solicitud de pase a producción

El comercio deberá completar la lista de test para solicitar la certificación y próximo paso a producción, los pasos a producción son los siguientes:

- El comercio debe completar su lista de test, todos los campos deben tener chequeado, mientras hacen sus pruebas cada ítem de la lista de test se marca en verde.
- Al tener la lista de test completamente chequeado, se habilita el botón de **"Solicitar certificación"**
- En el botón el comercio carga la url a certificar y un usuario/contraseña si se necesita para realizar las pruebas en su sitio
- El pedido de certificación llega al equipo de soporte, donde hacen compras de prueba en la url dada, si vemos que la integración se encuentra ok entonces se les da el acceso a producción.
- Se habilita una pestaña de producción donde el comercio tiene las claves de producción y puede configurar su perfil de aplicación en producción
- También el comercio debe cambiar las urls de las apis por las de producción.



Mejoras visuales sobre los formularios de catastros y pagos ocasionales

Como parte del objetivo de Bancard de ofrecer soluciones modernas, se han rediseñado los formularios de catastros y pagos ocasionales.

Estos nuevos formularios mantienen la capacidad de ser responsive y también recibieron nuevas opciones de personalización en el portal de comercios. Estas opciones se detallarán más adelante.

También es posible utilizar los nuevos temas proveídos por el portal de comercios, siendo todos ellos también personalizables por el comercio.

El requisito para poder implementar estos formularios es que se les dé un ancho **mínimo de 320px**. Se ha decido por este tamaño mínimo por ser un estándar de la industria.

Transición de los formularios a los nuevos estilos

Si el comercio ya cuenta con los formularios de pagos ocasionales o de catastros y los han personalizados, estas personalizaciones se mantendrán al lanzarse las mejoras visuales

El comercio podrá seguir operando de manera corriente y no es necesario realizar ajustes sobre la personalización o implementación de los formularios para poder seguir utilizándolos.

Formulario de catastro

El nuevo formulario de catastro se ha ajustado para poder incluir todos los datos necesarios del usuario en una sola vista. Esto es, el número de cédula se pedirá junto a los datos de la tarjeta y al hacer click sobre el botón de acción entonces ya se finalizará el proceso de catastro.

Vista previa del formulario de catastros – Tema por defecto


Tarjeta

Número de cédula de identidad

12345678

Número de tarjeta

1234 1234 1234 1234
CREDICARD

Fecha de Expiración

MM / AA

Código CVC

CVC


☐ Acepto los [términos y condiciones](#)

Siguiente





Formulario de pagos ocasionales

Si bien se ha modernizado la interfaz del formulario de pagos ocasionales, a nivel de flujos el formulario básicamente sigue siendo lo mismo. Es decir, se mejorado la legibilidad y usabilidad sin que esto pueda afectar a la funcionalidad del mismo.

La opción de pago con tarjetas se ha añadido, esto con miras a implementar otros métodos de pagos en el futuro.



Tarjeta

Número de tarjeta

1234 1234 1234 1234



Fecha de Expiración

MM / AA

Código CVC

CVC



Cuotas (sólo Paraguay)

1

Pagar (Gs. 946.631)



Nuevas opciones de personalización

Se han agregado varias nuevas opciones y herramientas con el fin de ayudar a la personalización de los formularios.

La cantidad de opciones a personalizar sobre los formularios ha aumentado y se agregaron las herramientas de **temas por defecto** y **una vista previa**.

Nueva vista de personalización

Personalización del formulario

Temas ▼

Pago Simple Registro de Tarjeta

Color fondo de campos
Color texto de campos
Color borde de campos
Color fondo del botón
Color texto del botón
Color borde del botón
Color fondo de formulario
Color del borde del formulario
Color fondo de encabezado
Color texto de encabezado
Color de línea separadora
Color texto de tu-eres-tu
Color texto de etiquetas

Card

Card number

1234 1234 1234 1234

Expiration

MM / YY

CVC

CVC

Click




Deshacer Copiar JSON

Favor verifica en tu Ecommerce que la combinación de tamaños y campos no altere el diseño en producción. Utilizar una fuente grande dentro de un área pequeña podría causar inconvenientes al mostrar el formulario.

Además de las personalizaciones existentes se han agregado las siguientes opciones:

Opción	Descripción	Valor por defecto
Color texto de tu-eres-tu	Opción que afecta a los colores del texto del enlace de Términos y condiciones	#555555
Color texto de etiquetas	Esto afecta a las etiquetas de los campos, tales como número de tarjeta, fecha de vencimiento o CVC.	#555555
Color del placeholder	Afecta a los textos de ejemplo que se encuentran en los campos. Por ejemplo, el del número de tarjeta es el texto 1234 1234 1234 1234.	#999999
Color mensajes de error	Este color será tomado por los mensajes de error en las validaciones de los campos. Están ocultos por defecto, por lo que para verificarlos es necesario enviar el formulario con datos incorrectos.	#b50b0b

Color icono CVV	Esto es para el color que tomará el icono que se encuentra del campo CVV.	#0a0a0a
Color principal de la pestaña	Aplica para el borde y texto de la pestaña de método de pago.	#5CB85C
Color de fondo de la Pestaña	Aplica para el color de fondo de la pestaña de método de pago.	#999999
Radio del borde de los inputs	Estilo del borde de los campos. Pueden tener los siguientes valores: - Redondeado - Píldora - Recto	Redondeado
Tamaño del texto del formulario	Ajuste el tamaño de todas las fuentes del formulario. Se debe de tener en cuenta de que colocar un tamaño de fuente grande puede ocasionar problemas al mostrar el formulario.	Normal
Fuente del texto del formulario	Ofrece un listado de fuentes posibles a ser utilizados para todos los textos del formulario.	
Posición de la etiqueta	La ubicación de las etiquetas puede variar entre dos posibles valores: -Flotante -Encima	Flotante

Vista previa

De manera a facilitar la edición de los formularios, se ha colocado la nueva herramienta de la vista previa en el perfil de aplicaciones del portal de comercio. La vista previa esta preparada para tomar de manera automática cada cambio de valor en las opciones del formulario y mostrar el ajuste sin que esto afecte a los formularios utilizados por los usuarios.

Los ajustes en la configuración, como siempre, aplicarán para los formularios de catastros y pagos ocasionales. Por lo tanto, se ofrece una vista previa de ambas formulario.

Los cambios se aplicarán automáticamente solo a la vista previa. Para trasladar los ajuste a los formularios, se deben de **confirmarlos** utilizando el botón **guardar**.

En caso de que se quieran volver atrás los ajustes no confirmados, se puede utilizar el botón **deshacer**.

También se tiene la opción de poder copiar un JSON con los estilos cargados en la vista previa.

Opciones para la vista previa

Personalización del formulario

Temas▼

Color texto del botón

Color borde del botón

Color fondo de formulario

Color del borde del formulario

Color fondo de encabezado

Color texto de encabezado

Color de línea separadora

Color texto de tu-eres-tu

Color texto de etiquetas

Color del placeholder

Color mensajes de error

Color icono CVV

Radio del borde de los inputs

Bordes rec▼

Pago Simple

Registro de Tarjeta

Tarjeta

Número de tarjeta

Expiración

CVC

Presionar

Pago seguro Bancard

PCI Security Standards Council

Deshacer

Copiar JSON

Recordar que los entornos de Sandbox y Producción cuentan con sus propios estilos y es importante **realizar los cambios dentro de la pestaña correspondiente**.

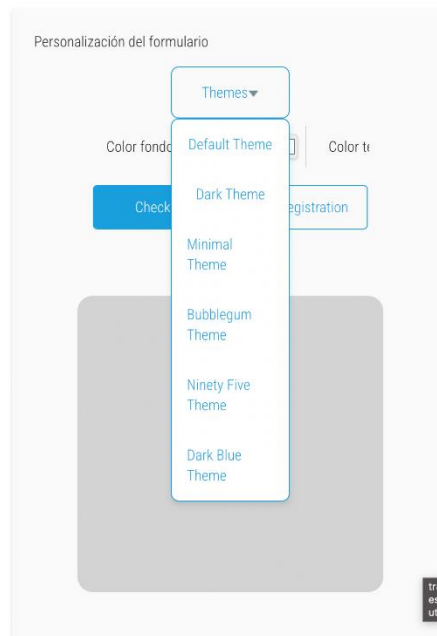
Temas predefinidos

Junto con la herramienta de vista previa, se tiene la opción de utilizar temas predefinidos como base. Al seleccionar uno de los temas, la vista previa tomará de manera automática los estilos asociados al tema.

80



Temas predefinidos disponibles



Es posible también utilizar estos temas predefinidos como base y sobre ellos realizar ajustes a los formularios.

Código de errores – Vpos 2.0

Código de errores Vpos 2.0	
Card errors (Código Errores para el catastro)	
CardAlreadyRegisteredByUserError	'The user has already registered the card.'
InvalidCiError	"The user's ci does not match with card's ci"
CardRequestAlreadyProcessedError	"The card request with process id #{@process_id} has already been processed."
CardInvalidDataError	'The data for the card is not correct.'
NewCardRequestNotFoundError	"New card request not found for process id: #{@process_id}"
CardNotFoundError	'The card does not exist'
CardAliasTokenExpiredError	'The card alias token has expired.'
CardBlockedError	'The card for the user is blocked.'
InvalidCardStatus	'The given status is incorrect'

Buy errors (Código de errores para pedido de pago)	
BuyNotFoundError	'Buy Not Found'
InvalidAmountError	"Amount attribute must be greater than zero."

Application errors (Código de errores para APIS vpos)	
ApplicationNotFoundError	'Application not found'
InvalidTokenError	'Invalid token'
InvalidPublicKeyError	'Invalid Public key'
PublicKeyNotFoundError	'Public key not found'
ApplicationCommunicationError	
ApplicationCredentialNotFoundError	"The credential for the application was not found."
CantCreateApplicationCredentialError	"The credential for the application could not be created."

Código de errores en los pagos	
0	APROBADA
1	LLAME CENTRO AUTORIZACION
2	CONSULTE SU EMISOR - CONDICION ESPECIAL
3	NEGOCIO INVALIDO
4	RETENGA TARJETA
5	NO APROBADO
6	ERROR DE SISTEMA
7	RECHAZO POR CONTROL DE SEGURIDAD
8	TRANSACCION FALLBACK RECHAZADA
9	SOLICITUD EN PROCESO
12	TRANSACCION INVALIDA
13	MONTO INVALIDO
14	TARJETA INEXISTENTE O INVALIDA
15	EMISOR INEXISTENTE,NO HABILITADO P/NEGOC
17	CANCELADO POR EL CLIENTE
19	INTENTE OTRA VEZ
21	NINGUNA ACCION A TOMAR
22	SOSPECHA DE MAL FUNCIONAMIENTO
30	ERROR DE FORMATO DE PAQUETE
33	TARJETA VENCIDA
34	POSIBLE FRAUDE - RETENGA TARJETA
35	LLAME PROCESADOR/ADQUIRENTE
36	TARJETA/CUENTA BLOQUEADA POR LA ENTIDAD
37	MES NACIMIENTO INCORRECTO-TARJ.BLOQUEADA
38	3 CLAVES EQUIVOCADAS - TARJETA BLOQUEADA
39	NO EXISTE CUENTA DE TARJETA DE CREDITO
40	TIPO DE TRANSACCION NO SOPORTADA
41	TARJETA PERDIDA - RETENGA TARJETA
42	NO APROBADO - NO EXISTE CUENTA UNIVERSAL

43	TARJETA ROBADA - RETENGA TARJETA
45	NO EXISTE LA CUENTA
46	EMISOR/BANCO NO RESPONDIO EN 49 SEGUNDOS
47	BCO.NEGOCIO O ATM NO RESPONDIO EN 49 SEG
49	OPERACION NO ACEPTADA EN CUOTAS
5C	TRN NO SOPORTADA-BLOQUEADA POR EL EMISOR
51	NO APROBADA-INSUF.DE FONDOS
52	NO APROBADA - NO EXISTE CUENTA CORRIENTE
53	NO APROBADA - NO EXISTE CUENTA AHORRO
54	TARJETA VENCIDA
55	CLAVE INVALIDA
57	TRANSACCION NO PERMITIDA
58	NO HABILITADA PARA ESTA TERMINAL
59	NO APROBADO - POSIBLE FRAUDE
60	NO APROBADO - CONSULTE PROCESADOR ADQUIR
61	EXCEDE MONTO LIMITE
62	NO APROBADA - TARJETA RESTRINGIDA
63	VIOLACION DE SEGURIDAD
65	EXCEDE CANTIDAD DE OPERACIONES
66	LLAMAR SEGURIDAD DEL PROCESADOR ADQUIR.
70	NEGOCIO EN MORA/FACTURA PENDIENTE
71	OPERACIÓN YA EXTORNADA
72	FECHA INVALIDA
73	CODIGO DE SEGURIDAD INVALIDO
75	EXCEDE LIMITE DE INTENTOS DE PIN - NO AP
77	EL CLIENTE NO TIENE CLAVE TODAVIA
78	CUENTA INACTIVA - PRIMER USO
79	CLAVE CAMBIADA
80	CLAVE ACTIVADA O CAMBIADA
81	ERROR CRIPTOGRAFICO EN EL PIN
82	ERROR DE AUTENTICACION
83	FRAUD/SECURITY - RESPUESTA DE LA MARCA
85	APROBADO - SIN IMPACTO FINANCIERO
86	IMPOSIBLE VERIFICAR PIN
87	TERMINADA LA RECONCILIACION DEL DIA
88	ERROR CRIPTOGRAFICO
89	POSICION FINANCIERA NO VERIFICABLE
9G	TARJETA BLOQUEADA-CONTACTE AL EMISOR
90	SWITCHER EN CIERRE - INTENTE EN 5 MINS.
91	EMISOR EN CIERRE - INTENTE OTRA VEZ
92	EMISOR DESCONECTADO - PROBLEMAS EN LINEA
93	NO APROBADO - VIOLACION DE LEY

94	TRANSACCION DUPLICADA - NO APROBADO
96	EMISOR DE LA TARJETA FUERA DE SERVICIO
98	BANCO DEL NEGOCIO/ATM FUERA DE SERVICIO
99	TRANSACCION EXTORNADA

Soporte para la integración

Ingresa al sitio: <https://comercios.bancard.com.py>, utilice la opción de menú Soporte, y el servicio vPOS.

Si se trata de una consulta de integración utilice la opción Pruebas de integración (desarrollo), estas consultas serán atendidas de lunes a viernes en horario de oficina.

Consideraciones Generales

- Existe un mecanismo de bloqueo que se activa cuando un comercio envía múltiples solicitudes de débito que son rechazadas consecutivamente. El control se activa cuando:

- En un lapso de 24hs la misma tarjeta cuenta con 7 intentos o reintentos de pago rechazados.
- En un lapso de 30 días la misma tarjeta cuenta con 35 intentos o reintentos de pago rechazados.

Recibe esta respuesta: MANDATO MC MAC 03 y 21 - INHABILITACIÓN 30 DIAS EN COMERCIO

Y la tarjeta queda bloqueada en el comercio por 30 días.